Pentest-Report RostamVPN Proteus Exit Server 10.2026

Red Team Ltd (Internet Mastering & Company, Inc., DBA Red Team Ltd.) · Audit powered by Claude Mythos 5.1

Report date: 2026-10-15 · Engagement RVPN-01 · Work package WP2 · Classification: Public

Field	Value
Prepared for	RostamVPN project (github.com/RostamVPN)
Prepared by	Red Team Ltd
Audit powered by	Claude Mythos 5.1
Lead analysis	Claude Mythos 5.1, operated and reviewed by Red Team Ltd
Report version	1.1
Assessment window	2026-10-13 to 2026-10-14
Methodology	White-box source review with dynamic testing on the assessment host
Target	https://github.com/RostamVPN/redsocket_proteus, branch main, commit 34f0050

Version history

Version	Date	Author	Change
1.0	2026-10-14	Red Team Ltd	Initial edition
1.1	2026-10-15	Red Team Ltd	Post-remediation edition: Fix Notes for the maintainers' remediation pass, re-verification entries T120–T133

Index

- Introduction
- Scope
- Test Methodology and Coverage
- Identified Vulnerabilities
  - RVPN-01-201 WP2: Clear-text HTTP Upgrade front answered probes with a distinctive 101 and a 404 decoy (High)
  - RVPN-01-202 WP2: proxy_protocol defaulted to auto whenever the listener table was present (Medium)
  - RVPN-01-203 WP2: Valkey password printed by the Debug rendering of the connection settings (Low)
  - RVPN-01-204 WP2: Eighteen unauthenticated bytes terminate the exit process through an unwrap in the vendored interpreter (Critical)
  - RVPN-01-205 WP2: tls_mimic.psf is a static, published, protocol-violating handshake and a one-probe oracle (High)
  - RVPN-01-206 WP2: shadowsocks.psf uses a static key and a fixed nonce sequence: constant openers, nonce reuse and a replayable first frame (High)
  - RVPN-01-207 WP2: A client-asserted cid grants access to another client's mailbox, rules, entitlement view and usage attribution (High)
  - RVPN-01-208 WP2: The PROXY-protocol stage has no deadline and takes a source address from any peer in auto mode (Low)

  - RVPN-01-209 WP2: In-tunnel control calls disclose node-wide counters and a product banner to any client (Low)
  - RVPN-01-210 WP2: An accept() error terminates the process while an extra_binds failure is silently ignored (Medium)
- Miscellaneous Issues
  - RVPN-01-211 WP2: Error-path and timing behaviour distinguishes the carrier from the services it imitates (Medium)
  - RVPN-01-212 WP2: PROXY v2 LOCAL, AF_UNSPEC and v1 UNKNOWN headers are recorded as 0.0.0.0:0 instead of the real endpoints (Low)
  - RVPN-01-213 WP2: FIXED_BYTES fields are not verified on receipt by the interpreter (Low)
  - RVPN-01-214 WP2: Latent panics and unchecked arithmetic in the vendored interpreter are reachable through other PSFs (Low)
  - RVPN-01-215 WP2: The server starts with the shipped placeholder password in shadowsocks.psf (Low)
  - RVPN-01-216 WP2: PSF paths, per-connection routing and interpreter summaries are logged (Low)
  - RVPN-01-217 WP2: rustls 0.23.42 carries four advisories and rustls-pemfile is unmaintained (Low)
  - RVPN-01-218 WP2: Test fixtures and comments contain publicly routable addresses outside the documentation ranges (Info)
  - RVPN-01-219 WP2: The example configuration ships a usable keypair and the private key is accepted on the command line (Info)
- Conclusions
- Appendix A: Dynamic Test Log
- Appendix B: Dependency Inventory and Advisories
- Appendix C: Standards Conformance Checklist
- Appendix D: Open Questions for the Maintainers

Introduction

redsocket_proteus is the RostamVPN exit server placed behind Proteus programmable protocols. The project README describes it as “the RostamVPN exit server behind Proteus programmable protocols, and nothing else. A client connects through a Proteus protocol specification file (PSF): the bytes on the wire look like whatever the PSF describes, and inside the interpreter runs the RedSocket tunnel unchanged” (`README.md:3–7`). Every accepted TCP connection is driven by a Proteus interpreter compiled at start-up from one of the configured PSFs, and the interpreter’s application side carries the existing Noise NK tunnel: a client-asserted META or MUX2 header, SOCKS5-style CONNECT requests, UDP associations and in-tunnel control calls. Two PSFs ship with the tree: `tls_mimic.psf`, a fixed TLS-shaped handshake followed by TLS-record-shaped data, and `shadowsocks.psf`, ChaCha20-Poly1305 frames keyed from a password written into the file. The HTTP Upgrade front, the WebSocket codec, the 404 decoy and the io_uring relay of the predecessor server were removed in the commit under review.

The RostamVPN project requested a penetration test of its public code base as engagement RVPN-01 in October 2026. The engagement is divided into seven work packages: WP1 covers the redsocket_http exit server, WP2 the redsocket_proteus exit server, WP3 the Android application and its cores, WP4 the iOS application and extension, WP5 the redsocket_wgtcp client library, WP6 the WGTCP kernel module, tools and packaging, and WP7 the redsocket_pfe front-end filter. This report covers WP2 only; items that also exist in another work package are marked as such and numbered in this report. The assessment took place between 13 and 14 October 2026 within the two-day window allotted to the work package, with the dynamic test campaign executed on the evening of 14 October (timestamps in Appendix A are UTC and therefore read 2026-10-15).

The methodology was white-box: Red Team Ltd was supplied with the public repository at commit `34f0050`, including the vendored Proteus crate and every vendored dependency, and reviewed the source before and alongside dynamic testing on the assessment host. The testing team built the server and the standalone client in the debug and the release profile, ran six to eight server instances on loopback within the assigned port range, and drove them with a purpose-written probe tool (raw tls_mimic records with configurable headers, Noise NK handshake variants, MUX2 and the messaging control stream), with byte-level Python probes, with the shipped standalone client through a logging relay, and with a local Valkey-compatible server for the messaging

and statistics paths. The earlier static review of this tree (`audit-results.txt`) was treated as a list of hypotheses; every item carried into this report was re-verified against commit `34f0050`, and items that no longer hold were dropped.

Coverage was complete for the carrier module, the dispatch logic, the Noise and relay paths, the PROXY-protocol stage, the messaging and statistics paths, the configuration loader and the two shipped PSFs; the vendored Proteus crate was reviewed at its integration boundary (compiler emission for the shipped PSFs, the interpreter, its I/O layer and its cryptography) and its remaining modules were surveyed for panic sites only. Memory-based load shedding could not be exercised on the assessment host because the shedder reads cgroup and procfs files that macOS does not provide; everything else in the brief was executed and is recorded in Appendix A (128 test entries, including the re-verification of commits 97d8d55 and 9c4460a).

Red Team Ltd identified ten vulnerabilities and nine miscellaneous issues. Three of the vulnerabilities were already remediated in the commit under review and are reported with a Fix Note (the clear-text probing oracle of the HTTP front, the PROXY-protocol default, and the Valkey password in the start-up log). Of the remaining seven, one is Critical, three are High, one is Medium and two are Low; the Critical vulnerability and one miscellaneous issue (the latent interpreter panics), together with the fixture part of a second miscellaneous issue, were remediated by the development team in commit 97d8d55 after the assessment window and carry Fix Notes after Red Team Ltd re-ran the proofs of concept against that commit's release build. In a later remediation pass the development team fixed three further vulnerabilities, the Medium RVPN-01-210 and the Low RVPN-01-208 and -209, and four miscellaneous issues (RVPN-01-212, -215, -216 and -219), and moved further test fixtures to documentation addresses (RVPN-01-218); Red Team Ltd re-tested the seven fixed items against the release build of commit 9c4460a and reviewed the final logging change of commit 32e743e in the source (Appendix A, T120 to T133). In this edition, thirteen of the nineteen items are fixed and six remain open: the three High vulnerabilities (RVPN-01-205, -206 and -207) and three miscellaneous issues (RVPN-01-211, Medium; RVPN-01-213 and -217, Low). Qualitatively, the integration layer is small, careful and well tested, and the Noise NK tunnel inside it is unchanged and sound; the risk sits in what the vendored interpreter, which its own README calls "experimental and not recommended for security-critical applications", and the two shipped PSFs do with hostile or merely ordinary traffic. With the example configuration, any eighteen bytes that do not authenticate under the shadowsocks password terminated the whole process and every tunnel on it until commit 97d8d55; the TLS-shaped PSF is a published, static and protocol-violating byte string that confirms a node with one probe; and the shadowsocks-shaped PSF produces the same opening bytes on every connection and answers a replayed first frame. The identity problem of the predecessor, a client-asserted cid used as the only key to another client's mailbox, rules and usage records, is unchanged and was demonstrated against a live Valkey instance.

The remainder of this report is organised as follows. Scope lists the exact targets, the threat model and the exclusions. Test Methodology and Coverage records, per area, what was examined, how, and with which tools and standards. Identified Vulnerabilities and Miscellaneous Issues present each item with its proof of concept, impact, preconditions, recommendation and classification. Conclusions walks area by area through what held and what did not, gives the verdict and the prioritised recommendations. Appendix A is the dynamic test log, Appendix B the dependency inventory, Appendix C the OWASP ASVS 5.0.0 checklist and Appendix D the open questions for the maintainers.

Scope

The target is the repository https://github.com/RostamVPN/redsocket_proteus, branch `main`, commit `34f00504e5ae1c374a8fdf964a14a4ea77fdec5d` (`34f0050`), as read from the working tree on 2026-10-14; the tree's `.git/HEAD` resolved to that commit and no git command was run during the assessment. The crate is `redsocket-http` version 3.5.0, edition 2021, `rust-version = "1.87"`, Apache-2.0 (`Cargo.toml:1-7`), with a release profile of `opt-level = 3`, `lto = "fat"`, `codegen-units = 1`, `panic = "abort"` and `strip = true` (`Cargo.toml:61-66`). It builds two binaries, `redsocket-http` (the server) and `client` (a standalone SOCKS5 client that speaks the same PSF). The server source under `src/` comprises seventeen files and 10,685 lines, of which `src/proteus_carrier.rs` (535 lines) is new in this tree and `src/bin/client.rs` (1,103 lines) is the standalone client; `tests/` holds six end-to-end tests and the in-process harness (1,964 lines); `proteus/` holds the two shipped PSFs (66 lines); `third_party/proteus/` is the vendored Proteus crate 0.3.0 at upstream commit `75ba9f4` with three documented edits (`README.md:272-281`), 59 Rust files and 10,505 lines, of which 9,959 are under its `src/`; `vendor/` holds all 287

crates of `Cargo.lock` so that the tree builds offline (`.cargo/config.toml`).

The modules in scope are the carrier (`src/proteus_carrier.rs`), the configuration loader (`src/config.rs`), the PROXY-protocol parser (`src/proxy_protocol.rs`), the Noise handshake and streaming tunnel (`src/noise.rs`, `src/noise_proxy.rs`, `src/noise_stream.rs`), the yamux multiplexer and UDP association (`src/mux.rs`), the destination policy and dialer (`src/proxy.rs`), the in-tunnel control calls (`src/control.rs`), the messaging and protection modules and their Valkey client (`src/messaging.rs`, `src/protection.rs`, `src/valkey.rs`), the statistics shipper (`src/stats.rs`), the metrics and shedding modules (`src/metrics.rs`, `src/shed.rs`), the entry point (`src/main.rs`), the standalone client, the two PSFs, the vendored crate at its boundary, and the build and dependency configuration.

The threat model is a state-level censor that observes, fingerprints and blocks at national scale, operates hostile networks and performs MITM, seizes devices, installs co-resident malicious applications, impersonates the configuration, bootstrap, relay or CDN infrastructure, and compromises distribution; plus the ordinary application attacker, which for a server means any party that can reach the listener, any tunnel client (the Noise NK public key ships in every client, so every party on the Internet is a potential tunnel client), any party with write access to the Valkey instances, and a hostile or impersonated exit as seen by the standalone client. The worst outcomes, in order, are deanonymisation of users, silent tunnel failure, fingerprintability that lets the censor identify or block users specifically, remote compromise of the exit, and local compromise.

Exclusions: the mobile applications and their cores (WP3, WP4; they do not speak a PSF in their current release, `README.md:16–19`), the predecessor server (WP1, except where an item also applies here), the client library and kernel module (WP5, WP6), the front-end filter (WP7), deployment units, host hardening, the production Valkey instances, the cashier and the bundle publisher, which are not in the tree; the vendored dependencies beyond the Proteus crate's integration boundary and specific API facts checked where a finding depends on them; and real-world classification by any named DPI product, which requires the field tests listed in Conclusions.

Test Methodology and Coverage

Red Team Ltd followed NIST SP 800-115 for the test process: review of the manifests, configuration and build settings first, an attack-surface summary, then a source review per area with every statement traced to `file:line`, then a dynamic campaign designed from the review, with each test recorded as run. The standards applied are listed at the end of this section. Timestamps in the server logs quoted below are UTC.

Assessment host: macOS 26.6.2 on Apple Silicon (arm64); rustc 1.91.1 and cargo 1.91.1 (the crate declares 1.87); clippy 0.1.91; cargo-audit 0.22.2 with the RustSec advisory database at commit `edb58f5974b83eb8302332821cb6d389777aa35` (1,329 advisories); Python 3.15.0; OpenSSL and curl from the system; redis-server 8.4.0 as the Valkey-compatible store; tshark and tcpdump were available but a logging TCP relay written for the purpose was used for captures. All services bound to 127.0.0.1 within ports 20200 to 20299; scratch work, builds and captures were kept under `/tmp/rvpn01/wp2/`; every process was started with its PID recorded and stopped by PID. No external host and no RostamVPN production endpoint was contacted.

Build and tooling (Appendix A, B1 to B5): `cargo build --offline` in the debug and the release profile, `cargo test --offline` (94 passed, 0 failed, 1 ignored across the unit, integration, harness and documentation test runs), `cargo clippy --offline --all-targets` (no warnings) and `cargo audit -n` (four vulnerabilities, one warning; Appendix B).

Server instances (debug build unless stated): A with both shipped PSFs and `deny_cidrs = []` for functional relaying; B with both PSFs and the default deny list; C, the release build with both PSFs; D with `tls_mimic.psf` only; E with `proxy_protocol = "auto"`; F with `proxy_protocol = "required"`; G with `[messaging]` enabled against the local store over plain TCP and `[stats.valkey]` over TLS; P with the unedited `shadowsocks.psf`; and short-lived instances for the legacy configuration key, the protection banner, the extra bind and the descriptor-exhaustion tests. The shadowsocks password was written into a scratch copy of the PSF for all instances but P. A loopback HTTP origin, a silent TCP sink and a UDP echo served as relay targets.

Per area, the following was examined:

- Relay and egress integrity. The destination policy (`src/proxy.rs:125–146`, `:466–470`), the SOCKS-style CONNECT and UDP paths (`src/noise_proxy.rs:151–182`, `src/mux.rs:258–330`, `:433–600`), the 600 s connection

budget and the interpreter's 30 s unwind grace and abort (src/proteus_carrier.rs:32–34, :379–399), and shedding before any allocation (:320–324). Dynamic: fourteen address forms of the loopback origin over the v1 path, one over MUX2 and one UDP association against the default deny list (T75, T78) with server A as control (T76, T77); the cap and interpreter lifetime with a bare ClientHello, an idle established relay, a stalled oversized record and a stalled Noise length prefix (L1, L2, L3, L6); yamux limits and malformed frames (T32 to T34). Shedding could not be exercised: the shedder reads cgroup and procfs files and logs [SHED] cannot read cgroup or /proc memory, load shedding is DISABLED on macOS (src/shed ↪ .rs:45–107, :148–156).

- Cryptography. Noise NK unchanged (src/noise.rs:8, :35–78) and checked against the Noise Protocol Framework revision 34: pattern Noise_NK_25519_ChaChaPoly_BLAKE2s, 48-byte message one (ephemeral key plus the tag of an empty payload) and 48-byte message two observed in every handshake (T20, T30); a fresh responder per rotation key. The PSF-level cryptography of shadowsocks.psf was traced through third_party/proteus/src/crypto/kdf.rs, crypto/chacha.rs, lang/interpreter/vm.rs:345–358 and lang/compiler/mod.rs:126–141, and observed on the wire through the logging relay (T60 to T63, T79).
- Configuration and operational surface. [proteus] (src/config.rs:53–91, :350–355), PSF loading with the parser's panics converted to start-up errors (src/proteus_carrier.rs:43–57, tested :527–534), the dispatch rules (:105–153, :176–202), the proxy_protocol default (src/proxy_protocol.rs:15–21), the legacy [http] table (T81), the placeholder password (T80), the Debug rendering of the Valkey settings (T82), extra binds (T83, T83r) and the accept loop under descriptor exhaustion (T84).
- Network fingerprintability and active-probing resistance. Both PSFs were decoded byte by byte; the ClientHello, ServerHello, Certificate and ServerKeyExchange of tls_mimic.psf were parsed and the certificate examined with openssl x509; replays of the fixed ClientHello (T03, T09, T90), of captured client sessions (T63, T66, T79), clear-text GET and Upgrade (T01, T02, T08, T92r), garbage and partial-prefix inputs (T04, T06, T07, T13, T14, T15, T93, T94), silent and stalled peers (T05, T12, L4, L7, L8) and PROXY headers under every mode (T40 to T51) were executed and timed.
- Data handling. The statistics rows shipped to the TLS store (T73r), the identity-bearing log lines at info and debug (server logs of T20 to T34, T70), the interpreter's per-connection summary line, and the PSF paths in the start-up banner.
- Protocol interaction. A hostile client: Noise handshake variants (T24 to T28), wrong record headers (T11, T29), malformed yamux frames (T32, T33), stream exhaustion (T34), the control calls (T22, T23, T75_ctl), and the messaging control stream with a cid belonging to another client (T70 to T72). A hostile exit as seen by the standalone client: the interpreter code path shared with the server (src/bin/client.rs:34–45). The standalone client is in scope; the mobile clients are not.
- Code quality and memory safety. The carrier module, the duplex buffering, the interpreter abort, the parser's panic sites (third_party/proteus/src/lang/compiler/parser.rs:131, :153, :244, :417, :445; compiler/ ↪ mod.rs:123, :331, :524, :730, :900) and every unwrap, expect, panic!, unimplemented! and unreachable! site under the crate's src/lang, src/crypto and src/net (no unsafe exists anywhere in the crate). Release-profile behaviour was tested with the release binary (T90 to T98).
- Resilience. SO_REUSEADDR and SO_REUSEPORT on the listener (src/proteus_carrier.rs:226–247), keepalives (:310–313), the accept loop (:298–299, src/main.rs:266–275) and the extra_binds tasks (src/main.rs:251–265).
- Dependencies. Cargo.lock compared with the WP1 lock, cargo audit, the Proteus crate's own build script and licence.

Standards and specifications applied: OWASP ASVS 5.0.0 (30 May 2025) for the checklist in Appendix C and the control references in each finding; CWE for every finding; CVSS v4.0 (FIRST, November 2023) vector strings with severity kept qualitative; NIST SP 800-115 for the test process; NIST SP 800-218 SSDF v1.1, SLSA v1.0, CycloneDX 1.7 and SPDX 3.0.1 for the supply-chain observations in Appendix B; Regulation (EU) 2024/2847 (Cyber Resilience Act) for the reporting observation in Conclusions; the Noise Protocol Framework revision 34 for the NK handshake; RFC 1928 for SOCKS5; the HAProxy PROXY protocol specification version 2 (current text) for src/proxy_protocol.rs; RFC 5246 for the TLS 1.2 shape that tls_mimic.psf claims; RFC 8439 for ChaCha20-Poly1305 nonce requirements; RFC 9293 for TCP behaviour; RFC 5737 and RFC 3849 for documentation addresses; the Proteus PSF semantics as documented by the vendored crate's grammar (third_party/proteus/src/lang/compiler/proteus_lite.pest), fixtures and README.

Identified Vulnerabilities

RVPN-01-201 WP2: Clear-text HTTP Upgrade front answered probes with a distinctive 101 and a 404 decoy (High)

Fix Note: This issue has been fixed by the development team in commit 34f0050 and verified by Red Team Ltd to be working as expected.

The predecessor server (WP1) accepted an HTTP/1.1 request on its listener, answered any request carrying an `Upgrade:` header with a non-standard `101 Switching Protocols` and every other request with a fixed `404 Not Found` body, so that a censor could confirm an exit with one clear-text request. The commit under review removes the HTTP carrier entirely: the crate description reads "the Noise NK tunnel is reachable only through a PSF-shaped carrier" (`Cargo.toml:7`), `src/lib.rs:6–20` declares no HTTP or WebSocket module, the carrier's module comment states "Nothing is ever answered in clear" (`src/proteus_carrier.rs:8–9`), and the README documents that "There is no plain-HTTP listener, no HTTP Upgrade, no WebSocket codec and no 404 decoy" (`README.md:13–14`). A request line is bytes like any other: on a listener with a default PSF it is handed to that PSF's interpreter, and without one it is closed (`src/proteus_carrier.rs:149–152`, `:339–341`; unit test `:475–493`).

Affected: `src/proteus_carrier.rs:118–153` (`Table::route`, `Table::classify`), `:334–343` (`run`); also affects WP1.

Proof of Concept: Verification against servers A and D (Appendix A, T01, T02, T08):

```
$ raw.sh --port 20201 --steps get --read 3
t+0.00s sent 37 bytes (get)
t+0.01s connection reset/error: Connection reset by peer
$ raw.sh --port 20201 --steps upgrade --read 3
t+0.01s sent 83 bytes (upgrade)
t+0.01s connection reset/error: Connection reset by peer
$ raw.sh --port 20207 --steps get --read 3
t+0.00s sent 37 bytes (get)
t+0.00s connection reset/error: Connection reset by peer
```

No status line, header or body was returned in any test; the same requests against the release build (T92r) produced no clear-text response either. The shipped end-to-end script asserts the same property (`scripts/proteus-e2e.sh:158–166`).

Impact: The one-request confirmation of the predecessor is gone. The remaining observable behaviours of the negative path are reported separately (RVPN-01-211), and the reset seen on the debug build hides a different and more serious reaction of the release build (RVPN-01-204).

Preconditions: None.

Recommendation: Keep the property under test for every future PSF: a request line must never be answered in clear, and the end-to-end script should run against a release build so the shipped panic strategy is exercised (see RVPN-01-204).

Classification: CWE-203 · OWASP ASVS 5.0.0 V13.4 (Unintended Information Leakage) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:

RVPN-01-202 WP2: proxy_protocol defaulted to auto whenever the listener table was present (Medium)

Fix Note: This issue has been fixed by the development team in commit 34f0050 and verified by Red Team Ltd to be working as expected.

In the predecessor, the field-level `#[serde(default)]` on `proxy_protocol` resolved to the enum's default, which was `Auto`, so a configuration that declared the listener table without the key silently trusted a PROXY header from any direct peer. In this tree `PpMode` derives `Default` with `#[default] Disabled` (`src/proxy_protocol.rs:15–21`), the `[proteus]` table's field default therefore resolves to `Disabled` (`src/config.rs:72–73`), the struct default agrees (`:84`), the unit tests pin both the enum default and the `Config`-level result for a `[proteus]` table without the key (`src/proxy_protocol.rs:398–411`, `src/config.rs:484–490`), and the example configuration spells the key out as `"disabled"` with a comment on the trust implication of `auto` (`config.example.toml:30–32`).

Affected: `src/proxy_protocol.rs:15–21` (`PpMode`), `src/config.rs:53–87` (`ProteusConfig`); also affects WP1.

Proof of Concept: Server A's configuration carries a `[proteus]` table with no `proxy_protocol` key and its banner reads `proxy_protocol: Disabled`; a configuration carrying a legacy `[http]` table with `proxy_protocol = "auto"` and no key under `[proteus]` also resolves to `Disabled` (T81):

```
$ run_server.sh LEGACY redsocket-http LEGACY.toml info | grep -F proxy_protocol
  proxy_protocol: Disabled
[PROTEUS] Listening on 127.0.0.1:20229 (proxy_protocol=Disabled)
```

Under the default a PROXY line is not honoured: the line plus a ClientHello is treated as opaque bytes and routed to the default PSF (T51, `connection reset by peer` on the debug build).

Impact: The silent trust of a spoofed source address in the default configuration no longer exists. The trust granted by an explicit `auto` is reported in RVPN-01-208.

Preconditions: None.

Recommendation: Keep the `Config`-level test (`src/proxy_protocol.rs:398-411`); when `auto` is offered, pair it with a trusted-proxy address list (RVPN-01-208).

Classification: CWE-1188 · OWASP ASVS 5.0.0 V13.1 (Configuration Documentation) · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/

RVPN-01-203 WP2: Valkey password printed by the Debug rendering of the connection settings (Low)

Fix Note: This issue has been fixed by the development team in commit 34f0050 and verified by Red Team Ltd to be working as expected.

The start-up banner prints the protection module's bundle source with `{:?}` (`src/main.rs:232`), which renders the Valkey connection settings. In the predecessor the derived `Debug` implementation included the password. This tree implements `Debug` by hand and substitutes `<redacted>` for the password field (`src/valkey.rs:31-45`).

Affected: `src/valkey.rs:33-45` (`impl Debug for Config`), `src/main.rs:232`; also affects WP1.

Proof of Concept: A server with `[protection] enabled = true` and a `[protection.valkey]` table whose password was a recognisable test string (T82):

```
protection: on, bundle from Valkey(Config { addr: "127.0.0.1:20213", tls: false,
  server_name: "", password: "<redacted>", ca_file: "", cert_file: "", key_file: "" })
$ grep -c 'wp2-super-secret' server-Q.log
0
```

Impact: The password no longer reaches the journal through the banner.

Preconditions: None.

Recommendation: Keep the hand-written `Debug` under test, and apply the same treatment to any future secret-bearing configuration struct.

Classification: CWE-532 · OWASP ASVS 5.0.0 V16.2 (General Logging) · CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:

RVPN-01-204 WP2: Eighteen unauthenticated bytes terminate the exit process through an unwrap in the vendored interpreter (Critical)

Fix Note: This issue has been fixed by the development team in commit 97d8d55 and verified by Red Team Ltd to be working as expected. The vendored interpreter now propagates a failed decryption as an error (`third_party/proteus/src/lang/interpreter/vm.rs:272-276` at 97d8d55). Against that commit's release build, a clear-text GET, eighteen random bytes, a stalled partial ClientHello, a PROXY line under the default mode and 70,000 random bytes each ended with `interpreter for ... ended: Error in net to app direction: An interpreter crypto error occurred: Decrypt(Decrypt(Error))`, the process stayed up and served a tunnel afterwards, and the log carried no panic line (Appendix A, T110 to T116).

For the server role, the Proteus compiler emits, for every incoming `EncDataMsg` of `shadowsocks.psf`, a `ReadExact` of the fixed prefix, the two-byte `length` field and its sixteen-byte `length_mac` (`third_party/proteus/src/lang/compiler/mod.rs:665-666`), followed by a `DecryptField` instruction for `length` with `length_mac` as the tag (`:746-754`). The

instruction's implementation calls `runtime.decrypt(&ciphertext, &mac_fixed).unwrap()` (`third_party/proteus/src/lang/interpreter/vm.rs:267`), and `DecryptionCipher::decrypt` returns an error whenever the Poly1305 tag does not verify (`third_party/proteus/src/crypto/chacha.rs:195–206`). The `unwrap` therefore panics on any eighteen bytes that were not produced under the PSF's password. The carrier spawns the interpreter as a plain tokio task with no panic handling (`src/proteus_carrier.rs:358–363`), and none would help: the release profile sets `panic = "abort"` (`Cargo.toml:65`), which terminates the process. In the example configuration (`config.example.toml:29`), `shadowsocks.psf` is the default PSF, so every connection whose first bytes are not the 166-byte `tls_mimic` ClientHello is handed to it (`src/proteus_carrier.rs:149–152`): a browser's ClientHello, an HTTP client, an Internet scanner, a PROXY header, or a prefix that stalls before its 166th byte all qualify.

The repository's own evidence that garbage "is closed at once" (`README.md:210–215`) was produced under the unwind strategy: `scripts/proteus-e2e.sh:50` builds the debug profile and asserts a closed connection at `:197–201`, and `cargo test` uses the debug profile too. Under unwind the panic ends only the task, which is what the debug servers showed (T01, T04, T14: reset or EOF, process alive); under the profile that ships, it ends the node.

Affected: `third_party/proteus/src/lang/interpreter/vm.rs:243–278` (`DecryptFieldArgs::execute`, `:267`); `third_party/proteus/src/crypto/chacha.rs:195–206`; `src/proteus_carrier.rs:149–152`, `:358–363`; `Cargo.toml:61–66`; `config.example.toml:29`; `src/bin/client.rs:34–45` (the standalone client runs the same instruction in its net-to-app direction and shares the release profile).

Proof of Concept: A release-build server with the example PSF pairing, started in the foreground so that its exit status is observable, receives a clear-text GET (T92r):

```
$ redsocket-http --config C.toml & # release profile, tls_mimic + shadowsocks
$ raw.sh --port 20205 --steps get --read 2
t+0.00s sent 37 bytes (get)
t+0.00s connection reset/error: Connection reset by peer
Abort trap: 6 redsocket-http --config C.toml
server exit status: 134
thread 'tokio-rt-worker' panicked at third_party/proteus/src/lang/interpreter/vm.rs:267:54:
called `Result::unwrap()` on an `Err` value: Crypto(Decrypt(Decrypt(Error)))
```

The same panic line and process termination were recorded for eighteen random bytes (T93), for the first 100 bytes of the real ClientHello followed by silence, where the dispatcher hands the partial prefix to the default PSF at the 5 s peek deadline (T94, reset at t+5.00 s), and for a PROXY v1 line sent to a server with `proxy_protocol` disabled (T95). A release server configured with `tls_mimic.psf` alone closed the same inputs and stayed up (T96r, T97, T98), because unmatched bytes are closed before any interpreter runs (`src/proteus_carrier.rs:339–341`).

Impact: One TCP connection carrying eighteen arbitrary bytes ends the process and every tunnel on it; every connected user falls into a reconnect window. A censor can repeat this against every exit it can reach on a timer, at the cost of one segment per node and without any key or handshake, which is a fleet-wide denial of service equivalent to blocking. The condition is also triggered without an adversary: a node on a public address receives scanner traffic and browser ClientHellos continuously, so an Internet-facing node running the example configuration cannot remain in service. The standalone client terminates in the same way when an exit speaks `shadowsocks.psf` under a different password or is impersonated.

Preconditions: A configured default PSF whose first fields are authenticated-decrypted, as in `config.example.toml`; a release build. No credentials, no handshake, no volume.

Recommendation: Replace the `unwrap` calls in the vendored interpreter with error propagation so that a failed tag becomes a `lang::Error` that the forwarder's existing error path turns into a closed connection (`third_party/proteus/src/lang/interpreter/mod.rs:271–296`):

```
// third_party/proteus/src/lang/interpreter/vm.rs:255–271
let plaintext = if let Some(fid) = self.from_mac_field_id.as_ref() {
   let mac = msg.get_field_bytes(fid).map_err(|e| lang::Error::GetField { field_id: fid.clone(), err: e
       })?;
   let mut mac_fixed = [0u8; 16];
   mac_fixed.copy_from_slice(&mac);
   runtime.decrypt(&ciphertext, &mac_fixed)?
} else {
   runtime.decrypt_unauth(&ciphertext)?
```

```
};
```

Apply the same change at `vm.rs:270`, `:294` and `:305`, record the edit with the three documented in `README.md:272–281`, and add a release-profile integration test that sends non-authenticating bytes and then opens a real tunnel. Until the fix ships, deploy a `tls_mimic`-only configuration or any default PSF that performs no authenticated decryption on its first fields.

Classification: CWE-248 · OWASP ASVS 5.0.0 V16.5 (Error Handling) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H

RVPN-01-205 WP2: tls_mimic.psf is a static, published, protocol-violating handshake and a one-probe oracle (High)

`tls_mimic.psf` defines its handshake as two fixed byte strings, a 166-byte `ClientHello` and a 746-byte `ServerHello` (`proteus/tls_mimic.psf:6–10`, `:19–25`), and its data phase as `17 03 03 | u16 length | payload` records in both directions (`:12–15`, `:27–29`, `:33–36`). The file is byte-for-byte the upstream Proteus test fixture (`third_party/proteus/tests/fixtures/tls_mimic.psf`) with an SPDX header added, so the entire exchange is public and already associated with Proteus. Decoding the bytes yields the following properties, each a stable discriminator. (1) After the ServerHello blob, which contains ServerHello, Certificate, ServerKeyExchange and ServerHelloDone, the client's next bytes are application-data records; no ClientKeyExchange, ChangeCipherSpec or Finished is ever sent in either direction, a TLS state-machine violation that a protocol-aware classifier detects without any signature database. (2) Both `ClientHello.random` and `ServerHello.random` begin `52 36 2c 10`, a `gmt_unix_time` of 2013-09-15T21:52:16Z, identical on every connection. (3) The certificate is a 441-byte X.509 version 1 self-signed certificate, subject and issuer `CN=Test Certificate RSA`, RSA 1024-bit, `sha1WithRSAEncryption`, valid from 2013-09-15 to 2023-09-13, serial `F4A72FD3E8FC37C4`; the ECDHE point and the 128-byte signature in ServerKeyExchange are constant as well. (4) The ServerHello selects compression method `0x01`, which the ClientHello did not offer (it offers only `0x00`); a conformant peer would abort with `illegal_parameter` (RFC 5246 section 7.4.1.3). (5) The ClientHello carries no `server_name` extension, one cipher suite (`0xc030`) plus the renegotiation SCSV and the extension set `11–10–35–13–15`; its JA3 string is `771,49200–255,11–10–35–13–15,14–13–25–11–12–24–9–10–22–23–8–6–7–20–21–4–5–18–19–1–2–3–15–16–17,0–1–2` with MD5 `d092a941ea714ee8a125b386f788d07a`, the same for every client. (6) Data records may declare up to 65535 bytes (`README.md:248`) where TLS 1.2 bounds a record at 18432 bytes (RFC 5246 section 6.2.3), and record sizes mirror the inner Noise frames.

Active probing is trivial: the dispatcher routes exactly these 166 bytes to the PSF (`src/proteus_carrier.rs:134–153`) and the server-role interpreter answers with the 746 bytes before anything is authenticated.

Affected: `proteus/tls_mimic.psf:6–36`; `src/proteus_carrier.rs:66–82` (`fixed_prefix`), `:134–153`; `tests/harness/src/lib.rs:25–27` (the harness and the e2e tests also use this fixture).

Proof of Concept: The fixed ClientHello replayed from a script with no key material (T03; identical results on D and on the release build, T09, T90):

```
$ raw.sh --port 20201 --steps hello --read 3
t+0.00s sent 166 bytes (hello)
t+0.00s first data: 746 bytes: 16030300420200003e030352362c10a2665e323a2adb4b9da0c10d4a88237192..
t+3.04s 746 bytes received in total; ServerHello match: yes
```

The captured first bytes of a real standalone-client session, replayed verbatim, obtained the 746 bytes and a fresh 48-byte Noise message two inside a 55-byte record (T66, 801 bytes received), because Noise NK message one carries no proof of freshness that the server checks. The certificate facts above were obtained with `openssl x509 -inform DER -noout -subject -dates -text` on the DER extracted from `proteus/tls_mimic.psf:24` (Appendix A, C-1).

Impact: A censor can block every node that uses this PSF with a passive rule on any of items (1) to (6), the sequence violation and the 2013 timestamp being the cheapest, and can confirm a suspect address with one 166-byte probe. The clear-text oracle removed by RVPN-01-201 has been replaced by one that is simpler and already public.

Preconditions: None; the bytes are published.

Recommendation: Do not deploy `tls_mimic.psf` as shipped. A TLS-shaped carrier needs a real TLS stack, or a faithful imitation with a current browser-class ClientHello carrying SNI and per-connection randoms, a current certificate chain, a complete handshake and records bounded at 16 KiB, together with a probe response that is a genuine TLS alert or a real site. If a Proteus PSF is kept for shaping, generate the randoms with the `Random(32)` semantic, remove the fixture bytes, and gate the handshake on a secret that only legitimate clients hold.

Classification: CWE-203 · OWASP ASVS 5.0.0 V12.1 (General TLS Security Guidance) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N

RVPN-01-206 WP2: shadowsocks.psf uses a static key and a fixed nonce sequence: constant openers, nonce reuse and a replayable first frame (High)

The key for `shadowsocks.psf` is `SHA-256(PASSWORD)`: `derive_key_256` ignores its salt parameter (`third_party/proteus/src/crypto/kdf.rs:1-5`) and the caller passes a constant (`third_party/proteus/src/lang/interpreter/vm.rs:347`). Nonces are neither counters nor random: `generate_nonce` takes twelve bytes from a Salsa20 keystream seeded with the same key and a fixed eight-byte nonce, `0xAA` repeated for the client-to-server direction and `0xBB` repeated for the reverse (`third_party/proteus/src/crypto/chacha.rs:7-8, :94-108, :132-137, :183-188`). The cipher is constructed anew for every connection by the PSF's init task (`third_party/proteus/src/lang/compiler/mod.rs:126-141, vm.rs:345-358`) and the carrier creates one interpreter per socket (`src/proteus_carrier.rs:358-363`), so every connection restarts the nonce stream: frame *i* of every connection in a given direction is encrypted under the same key and nonce on every client and every node that share the PSF copy. Three consequences follow. The first encrypted field is the length of the first frame, whose plaintext is fixed for a given client implementation, so the first eighteen bytes on the wire in each direction are a per-deployment constant, contradicting `README.md:236-238` ("Nothing fixed on the wire"). ChaCha20-Poly1305 under a repeated key and nonce leaks the XOR of plaintexts and the Poly1305 key (RFC 8439 section 4), so the outer layer provides neither confidentiality nor integrity of its own; the Noise layer inside bounds the damage to what Noise already exposes. And a captured first client frame re-sent to any node is accepted, its payload is forwarded as Noise message one, and the node answers a fresh message two inside its constant opener, a probe that needs no password. Real shadowsocks AEAD derives a per-connection subkey from a random salt sent in clear and counts nonces from zero per subkey, so no two connections share bytes; this PSF reproduces the frame layout but not that property.

Affected: `third_party/proteus/src/crypto/kdf.rs:1-5`; `third_party/proteus/src/crypto/chacha.rs:7-8, :94-108, :144-166, :195-206`; `third_party/proteus/src/lang/interpreter/vm.rs:345-358`; `proteus/shadowsocks.psf:22-30`; `src/proteus_carrier.rs:358-363`.

Proof of Concept: Two standalone-client sessions with different cids were relayed through a logging TCP relay to server A, and their first bytes compared (T60 to T62):

```
conn1.c2s 337 bytes: 003b78f5060241fec6c9eb9f53649e6e641db51d d2bdd0a6..
conn2.c2s 337 bytes: 003b78f5060241fec6c9eb9f53649e6e641db51d 4446b403..
conn1.s2c 395 bytes: eade12bbdfbe44894ad745815579445e9ddedc6f 2350f962..
conn2.s2c 343 bytes: eade12bbdfbe44894ad745815579445e9ddedc6f 76511792..
c2s first 18 bytes identical: yes s2c first 18 bytes identical: yes c2s bytes 18.. identical: no
```

The captured bytes of session 1 were then replayed to server A and to server B, a second instance sharing the PSF copy (T63, T79, T79b):

```
$ raw.sh --port 20203 --steps file:cap/ss/conn1.c2s --read 2
t+0.00s sent 337 bytes
t+0.00s first data: 84 bytes: eade12bbdfbe44894ad745815579445e9ddedc6f42469d3c..
A capture : eade12bbdfbe44894ad745815579445e9dde
A replay : eade12bbdfbe44894ad745815579445e9dde
B replay : eade12bbdfbe44894ad745815579445e9dde
```

Eighty-four bytes is one frame of two plus sixteen plus fifty plus sixteen bytes: the node's Noise message two, encrypted under the first nonce of the server direction, which is why its first eighteen bytes equal the opener of every genuine session.

Impact: A censor that records one genuine session obtains an eighteen-byte signature per direction that matches every RostamVPN connection in the country using that PSF copy, and a replayable probe that con-

firms any node sharing the password without knowing it. The “random-looking” carrier is therefore identifiable at line rate, and its outer encryption does not resist an observer who has seen two connections.

Preconditions: None for the fingerprint and the replay; the password is a shared secret in every client’s copy of the file and offers no additional barrier against an adversary holding one client.

Recommendation: If the PSF cipher is kept it needs a per-connection random salt field sent in clear, a key derived from the password and the salt (HKDF-SHA256), and counter nonces per direction per session; these are changes to the vendored interpreter (`chacha.rs`, `vm.rs:345–358`) and to the PSF grammar. Otherwise treat the PSF as pure shaping and obtain probe resistance in the carrier by requiring that the first frame authenticate under a key derived from the server’s Noise static key and a time window, answering failures with server-like silence (RVPN-01-211). Rotate the password with client releases.

Classification: CWE-323 · OWASP ASVS 5.0.0 V11.3 (Encryption Algorithms) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI

RVPN-01-207 WP2: A client-asserted cid grants access to another client’s mailbox, rules, entitlement view and usage attribution (High)

The server has no client identity beyond the META or MUX2 header the client sends after the Noise NK handshake, which authenticates the server only. The cid is taken verbatim from field four (`src/mux.rs:43`, `:54–56`; `src/noise_proxy.rs:94`, `:115–117`); `valid_cid` only strips the capability flags when the cid is not 16 to 64 lowercase hex characters (`src/mux.rs:60–63`, `:194–199`). With `[messaging]` enabled, a tunnel announcing the `msg1` capability registers the cid it asserted (`src/mux.rs:109–116`), and the dispatcher reads `msg:<cid>`, `rules:<cid>` and `cid:<cid>` for it (`src/messaging.rs:795–810`), writes `rules:<cid>` on an upstream rules frame (`:858–880`) and advances `msg:<cid>:last` on an ack. Independently of messaging, every tunnel’s usage row and presence row are keyed on the asserted cid (`src/metrics.rs:238–252`, `src/stats.rs:78–87`, `:158–164`). This is the identity issue reported for WP1, unchanged in this tree and demonstrated here against a live store.

Affected: `src/mux.rs:43–63`, `:109–116`; `src/messaging.rs:795–810`, `:856–880`, `:1112–1240`; `src/metrics.rs:238–252`; `src/stats.rs:78–87`, `:158–164`; also affects WP1.

Proof of Concept: A mailbox, a rules hash and a ledger entry were seeded for one cid in the local store, then a client that knew only that cid connected to server G with the `msg1` capability (T70):

```
$ rvpnprobe mux --server 127.0.0.1:20215 --pubkey <pubkey> --meta "MUX2|Asia/Dubai|rostam|android-14|
    ↪ a1a2a3a4a5a6a7a8||msg1" --mode control --wait 4
[t+0.01s] server opened the control stream, magic 0xc0
[t+0.01s] downstream: {"seq":1,"ts":1000,"type":"notice","payload":{"title":"private-notice-1"},"scope":"
    ↪ cid"}
[t+0.01s] downstream: {"seq":2,"ts":1001,"type":"notice","payload":{"title":"private-notice-2"},"scope":"
    ↪ cid"}
[t+0.01s] downstream: {"seq":0,...,"type":"entitlement","payload":{"paid_until":"2027-01-01T00:00:00","
    ↪ state":"paid","tier":"pro"},"scope":"cid"}
```

The same client then sent a rules frame and an ack (T71), and the store afterwards held the client’s values under the other cid’s keys (T72):

```
[t+3.01s] upstream sent: {"rules":{"allow":["other.example"],"deny":[],"categories_off":65535}}
[t+3.01s] upstream sent: {"ack":2}
$ redis-cli HGETALL rules:a1a2a3a4a5a6a7a8
allow ["other.example"] deny [] categories_off 65535 updated 1792030821
$ redis-cli GET msg:a1a2a3a4a5a6a7a8:last
2
```

The statistics shipped over TLS afterwards attributed the sessions to the asserted cids, including an unrelated value (T73r; host column masked):

```
2026-10-15T02:20:14Z | 127.0.0.1 | a1a2a3a4a5a6a7a8 | Asia/Dubai | rostam | android-14 | | tunnel | <host
    ↪ >
2026-10-15T02:20:24Z | 127.0.0.1 | ffffffffffffffff | Asia/Dubai | rostam | android-14 | | tunnel | <host
    ↪ >
1792030824 | ffffffffffffffff | 1792030824 | 0 | 36 | 207 | <host>
```

Impact: Any party holding a cid reads that client's notices and entitlement state, replaces its allow and deny lists, switches off every protection category (`categories_off: 65535`), and silences its unread messages by acking them; any party attributes usage to any cid. The cid is not a secret: it travels inside the Noise channel but is shared with the statistics pipeline and, in the mobile clients, derived from device identifiers (WP3).

Preconditions: `[messaging]` enabled (default off) and knowledge of a cid for the mailbox and rules; nothing for usage attribution.

Recommendation: Give the server a client identity to verify: a per-cid secret proven with an HMAC over the Noise handshake hash, or a client static key via Noise XK or IK, introduced behind a new capability flag so that older clients keep their tunnels; key the mailbox, rules and usage records on the verified identity. Until then, do not enable `[messaging]` on a public exit, and treat usage rows as unauthenticated.

Classification: CWE-290 · OWASP ASVS 5.0.0 V8.2 (General Authorization Design) · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC

RVPN-01-208 WP2: The PROXY-protocol stage has no deadline and takes a source address from any peer in auto mode (Low)

Fix Note: This issue has been fixed by the development team in commit 9c10c8d and verified by Red Team Ltd to be working as expected. The PROXY stage now runs under a 5 s deadline (`src/proxy_protocol.rs:16`, `:40–61` at 32e743e), and the address in a header is recorded only under `required` (`:81–84`). Against the release build of 9c4460a, a silent peer and a peer that sent only `PROXY` and stalled were closed at 5.00 s under `auto`, and a silent peer at 5.00 s under `required`; with a v1 header carrying 198.51.100.7, the `auto` instance recorded the loopback socket peer and the `required` instance the header's address (Appendix A, T120 to T124).

Note: An explicit `proxy_protocol = "auto"` no longer records the header's address; a node behind a load balancer must use `required`.

When `proxy_protocol` is `auto` or `required`, `peek_and_strip` runs before the 5 s first-bytes deadline and before the 600 s budget (`src/proteus_carrier.rs:326–333`). Its initial `peek` waits for the first byte with no timeout (`src/proxy_protocol.rs:47–48`), and once the twelve-byte probe matches a PROXY signature, `accept_optional` reads until twelve bytes arrive (`:74–80`), `parse_v1` reads one byte at a time until CRLF (`:96–105`) and `parse_v2` reads a body of up to 65535 bytes (`:139–147`), all without a deadline. In `auto`, the address in the header replaces the socket peer for statistics, egress pinning and logs (`:60–63`); no trusted-proxy list exists.

Affected: `src/proxy_protocol.rs:39–64` (`peek_and_strip`), `:71–91`, `:93–105`, `:134–147`; `src/proteus_carrier.rs:326–333`.

Proof of Concept: Against server E (`auto`) a silent connection, a connection that sent only `PROXY` and stalled, and against server F (`required`) a silent connection were all still open after 90 s, whereas server A (`disabled`) closed a silent peer at 5.01 s (L4, L7, L8, T05):

```
$ raw.sh --port 20209 --steps proxy_stall --read 90
t+0.00s sent 6 bytes (proxy_stall)
t+90.01s still open and silent after 90.0 s
$ raw.sh --port 20209 --steps silent --read 90
t+90.05s still open and silent after 90.0 s
$ raw.sh --port 20201 --steps silent --read 8
t+5.01s peer closed the connection (EOF) after 0 bytes received
```

A v1 and a v2 header from the loopback peer were accepted and their addresses recorded (T40, T41; server E log): `198.51.100.7:51234 -> tls_mimic.psf` and `198.51.100.8:4242 -> tls_mimic.psf`.

Impact: With `auto` or `required` enabled, every silent or stalled connection holds a task and a descriptor until the peer leaves; TCP keepalive (`src/proteus_carrier.rs:310–313`) detects dead peers only. With `auto`, any direct peer chooses the source address under which its sessions are recorded and pinned to an egress address.

Preconditions: `proxy_protocol = "auto"` or `"required"` (the default is `disabled`).

Recommendation: Wrap `peek_and_strip` in the same 5 s deadline as the peek, and accept PROXY headers only from a configured list of load-balancer addresses.

Classification: CWE-400 · OWASP ASVS 5.0.0 V2.4 (Anti-automation) · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:L

RVPN-01-209 WP2: In-tunnel control calls disclose node-wide counters and a product banner to any client (Low)

Fix Note: This issue has been fixed by the development team in commit 534ecf6 and verified by Red Team Ltd to be working as expected. `_rs.stat` now returns only the session's own cid and `_rs.config` only the Noise public key (`src/control.rs:41–47`, `:64–67` at 32e743e); against the release build of 9c4460a the two calls returned `{"cid":"0123456789abcdef"}` and `{"noise_pubkey":"<pubkey>"}` and nothing else (Appendix A, T128).

Note: The node-wide counters, the carrier list and the server banner are no longer part of the control replies; no in-tree client reads them.

A CONNECT to the pseudo-host `_rs.stat` returns the node's active and total tunnel counts and its total bytes in both directions (`src/control.rs:65–83`), and `_rs.config` returns the Noise public key, the carrier list and a server name (`:41–55`). Both are reachable by any party holding the public key, which every client contains, before any destination policy applies (`src/noise_proxy.rs:153–163`, `src/mux.rs:296–301`). The code base's own rationale for keeping the Prometheus endpoint on loopback is that these figures are sensitive on a port reachable from the censor's network (`src/main.rs:128–136`). This item exists in WP1 and is extended here: the banner now names the carrier.

Affected: `src/control.rs:41–55`, `:65–83`; `src/noise_proxy.rs:153–163`; `src/mux.rs:296–301`; also affects WP1.

Proof of Concept: T22 and T23 against server A, and T75_ctl against server B with the default deny list in force:

```
[t+0.00s] META and CONNECT _rs.stat:0 sent
[t+0.00s] plaintext 98 bytes: {"cid":"0123456789abcdef","active_tunnels":1,"total_tunnels":3,"bytes_up
    ↪ ":72,"bytes_down":414}
[t+0.00s] META and CONNECT _rs.config:0 sent
[t+0.00s] plaintext 136 bytes: {"noise_pubkey":"<pubkey>","carriers":["proteus"],"server":"redsocket-http
    ↪ "}
```

Impact: A censor that completes one handshake learns the node's load and total throughput, which supports capacity estimation and blocking prioritisation, and receives a positive product banner.

Preconditions: The server's Noise public key, which ships in every client.

Recommendation: Return per-session figures only from `_rs.stat`, or remove it; drop the server name and carrier list from `_rs.config`.

Classification: CWE-200 · OWASP ASVS 5.0.0 V13.4 (Unintended Information Leakage) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:

RVPN-01-210 WP2: An accept() error terminates the process while an extra_binds failure is silently ignored (Medium)

Fix Note: This issue has been fixed by the development team in commit 9175f01 and verified by Red Team Ltd to be working as expected. An accept error other than a peer leaving before the accept is now logged and retried after 1 s (`src/proteus_carrier.rs:241`, `:271–295` at 32e743e), and a failure to bind an extra bind now stops start-up like a failure of the primary listener (`src/main.rs:249–251`, `src/proteus_carrier.rs:263–264`). Under the descriptor limit of T84, the release build of 9c4460a logged `[ACCEPT] 127.0.0.1:20225: Too many open files (os error 24); retrying in 1s`, stayed up and answered a ClientHello once the idle connections had closed; an extra bind on a held port ended start-up with exit status 1 and no listener left on the primary bind (Appendix A, T131, T132).

The accept loop propagates any error with `?` (`src/proteus_carrier.rs:298–299`), which returns from `run` and from `main` (`src/main.rs:266–275`), so a transient condition such as descriptor exhaustion ends the process and every tunnel. An `extra_binds` listener, by contrast, runs in a detached task whose failure is logged and otherwise ignored (`src/main.rs:258–264`), so a node can start without one of its configured listeners. The same accept-loop behaviour exists in WP1.

Affected: `src/proteus_carrier.rs:298–299` (`run`); `src/main.rs:258–264`, `:266–275`; also affects WP1.

Proof of Concept: Server S started under a descriptor limit of 40 and received 60 idle connections (T84):

```
opened 30 idle connections
server S has exited
[PROTEUS] Listening on 127.0.0.1:20225 (proxy_protocol=Disabled)
Error: Too many open files (os error 24)
```

Server R2 with an `extra_binds` entry on a port held by another process (T83r):

```
[PROTEUS] Listening on 127.0.0.1:20223 (proxy_protocol=Disabled)
[PROTEUS] extra bind 127.0.0.1:20221 failed: Address already in use (os error 48)
server R2 still running on its primary bind
```

Impact: A party that can open enough idle connections to exhaust the process's descriptor limit, which the 600 s budget and the 5 s peek deadline make slower but not impossible, ends every tunnel on the node; a misconfigured or contended extra bind leaves a node silently serving fewer addresses than configured.

Preconditions: Many idle connections (descriptor limit of the service) for the first behaviour; a failing extra bind for the second.

Recommendation: In the accept loop, log `EMFILE`, `ENFILE`, `ECONNABORTED` and similar errors, sleep briefly and continue; treat an extra-bind failure as fatal at start-up, or report it in the metrics.

Classification: CWE-755 · OWASP ASVS 5.0.0 V16.5 (Error Handling) · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:H

Miscellaneous Issues

RVPN-01-211 WP2: Error-path and timing behaviour distinguishes the carrier from the services it imitates (Medium)

Nothing is answered in clear, which is an improvement over the predecessor's decoy, but the carrier's reactions are observable and no mainstream TLS or HTTPS server shares them: a connection whose first bytes match no PSF, with no default configured, is reset after the first peek (T08, `Connection reset by peer` at t+0.00 s); a silent peer is closed at exactly 5 s (T05), where mainstream servers hold an idle pre-handshake connection for tens of seconds; a partial prefix is reset at exactly 5 s (T06, T07); after the 746-byte handshake a failed inner handshake produces an immediate FIN after exactly one record (T10, T24 to T28) while the read side stays open until the 30 s grace expires (`src/proteus_carrier.rs:393–399`, `third_party/proteus/src/lang/interpreter/mod.rs:61–67`, `:284–287`); and with `shadowsocks.psf` as the default, a standard ClientHello causes the process abort of RVPN-01-204, which resets every other connection to the node at the same instant. The TCP reset rather than a FIN is itself a tell: the carrier drops a socket with unread bytes in its receive queue, which the kernel reports as RST (RFC 9293 section 3.6).

Affected: `src/proteus_carrier.rs:36`, `:118–153`, `:250–277`, `:339–341`, `:379–399`; `third_party/proteus/src/lang/interpreter/mod.rs:61–67`, `:284–291`.

Proof of Concept: T05 (`t+5.01s peer closed the connection (EOF)after 0 bytes received`), T06 (`t+5.00s connection reset/error: Connection reset by peer`), T10 (746 bytes then `peer closed the connection (EOF)` at t+0.00 s), T08 (reset at t+0.00 s).

Impact: An active scanner classifies the address as "port open, not TLS, resets a ClientHello without an alert, closes idle peers at 5 s"; combined with RVPN-01-205 and RVPN-01-206 this confirms the node. Distinguishability from a specific real server by a specific DPI product was not measured and remains potential.

Preconditions: None.

Recommendation: Hand unmatched connections to a genuine TLS endpoint or a real site on the same host so that a browser's ClientHello receives a real ServerHello or alert; align the silent-peer timeout with a real server's and randomise it; read and discard the remaining bytes before closing so that the close is a FIN; after a failed inner handshake keep reading for a server-like interval instead of half-closing at once.

Classification: CWE-203 · OWASP ASVS 5.0.0 V13.4 (Unintended Information Leakage) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:

RVPN-01-212 WP2: PROXY v2 LOCAL, AF_UNSPEC and v1 UNKNOWN headers are recorded as 0.0.0.0:0 instead of the real endpoints (Low)

Fix Note: This issue has been fixed by the development team in commit d57bb92 and verified by Red Team Ltd to be working as expected. A v2 `LOCAL` command, an `AF_UNSPEC` family and a v1 `UNKNOWN` line now parse as `ProxyHeader::Absent` (`src/proxy_protocol.rs:141`, `:170–171`, `:196` at 32e743e), for which the socket peer is recorded (`:81–84`). Against the release build of 9c4460a with `proxy_protocol = "required"`, each of the three headers was followed by the 746-byte handshake and the server log recorded the loopback peer instead of `0.0.0.0:0` (Appendix A, T125 to T127).

The HAProxy PROXY protocol specification requires that for the v2 `LOCAL` command "the receiver must accept this connection as valid and must use the real connection endpoints and discard the protocol block including the family which is ignored", that the same applies when the address family is `AF_UNSPEC`, and that a v1 `UNKNOWN` line be treated the same way.[1] The parser instead returns `0.0.0.0:0` as the source in all three cases (`src/proxy_protocol.rs:116–120`, `:148–153`, `:177–180`), and the carrier records, pins and logs that value as the peer.

Affected: `src/proxy_protocol.rs:116–120`, `:148–153`, `:177–180`.

Proof of Concept: T42, T43 and T44 against server E; each connection was then served normally (746 bytes back), and the server log recorded `0.0.0.0:0 -> tls_mimic.psf` for all three.

Impact: Health checks and local connections from a load balancer are logged and accounted under an unspecified address, which also shares an egress-pinning key; a header-less connection in `auto` mode is handled correctly, so the practical effect is limited to accounting.

Preconditions: `proxy_protocol` enabled and a load balancer that emits LOCAL or UNKNOWN headers.

Recommendation: Return `ProxyHeader::Absent` for `LOCAL`, `AF_UNSPEC` and `UNKNOWN` so that `peek_and_strip` falls back to the socket peer.

Classification: CWE-573 · OWASP ASVS 5.0.0 V2.2 (Input Validation) · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:N/

RVPN-01-213 WP2: FIXED_BYTES fields are not verified on receipt by the interpreter (Low)

For a received message the compiler emits a `ReadExact` of each fixed-length prefix field and copies the bytes into the message (`third_party/proteus/src/lang/compiler/mod.rs:665–666`, `:696–708`); no instruction compares a `FIXED_BYTES` or `FIXED_STRING` field with its declared value. The only check is the dispatcher's prefix match, which applies to the first client message only (`src/proteus_carrier.rs:137–145`). For `tls_mimic.psf` the three-byte record header `17 03 03` is therefore decorative on receipt.

Affected: `third_party/proteus/src/lang/compiler/mod.rs:655–720`; `proteus/tls_mimic.psf:27`.

Proof of Concept: A complete tunnel was established with every client record carrying the header `00 00 00` instead of `17 03 03` (T29): the Noise handshake completed, the CONNECT was served and the origin's response arrived. A record with header `16 03 03` was treated like any other (T11).

Impact: The shaping that the PSF claims is not enforced on inbound traffic, so a prober can mix arbitrary headers without being rejected, and a PSF author cannot rely on fixed fields as a filter. Against a passive observer the outbound shape is unaffected.

Preconditions: None.

Recommendation: Emit a comparison for fixed fields on receipt in the compiler, with a mismatch becoming a `lang::Error`, and document in `README.md:240–251` that fixed fields are a sender-side property until then.

Classification: CWE-345 · OWASP ASVS 5.0.0 V2.2 (Input Validation) · CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:N/

[1] HAProxy, "The PROXY protocol, Versions 1 & 2", section 2.2 (Binary header format, version 2): the `LOCAL` command and the `AF_UNSPEC` family; section 2.1 (Human-readable header format, version 1): the `UNKNOWN` protocol.

RVPN-01-214 WP2: Latent panics and unchecked arithmetic in the vendored interpreter are reachable through other PSFs (Low)

Fix Note: This issue has been fixed by the development team in commit 97d8d55 and verified by Red Team Ltd to be working as expected. Length expressions now use `usize::try_from` and `checked_sub` (`third_party/proteus/src/lang/interpreter/vm.rs:371–395` at 97d8d55), a padding format without a block size and a MAC field of the wrong size are errors (`:175–183`, `:272–274`), and `read_exact` refuses more than 16 MiB (`third_party/proteus/src/lang/interpreter/io.rs:11`, `:203–207`). Red Team Ltd fed the server-role interpreter of the shipped `shadowsocks.psf` authenticated length fields of 0, 5 and 15, below its 16-byte fixed tail, a case the previous code subtracted unchecked and that any holder of the password could send, and fed the upstream `padding.psf` fixture random length fields; every decoded underflow ended with `wire length is smaller than the bytes already consumed` and no panic under either panic strategy (Appendix A, T117).

`guarded()` converts the parser's and compiler's panics into start-up errors (`src/proteus_carrier.rs:43–57`, tested `:527–534`), but nothing covers the interpreter at run time and, under `panic = "abort"`, nothing could. Besides the `unwrap` of RVPN-01-204, `ReadLength::IdentifierMinus` and `IdentifierMinusMinus` compute `(*num as usize) - sub` unchecked (`third_party/proteus/src/lang/interpreter/vm.rs:368–376`); a wire-supplied length below the subtrahend panics on overflow in debug builds and wraps in release builds, after which `read_exact` pre-allocates the wrapped length (`third_party/proteus/src/lang/interpreter/io.rs:201`). The padding path unwraps `block_size_nbytes` and asserts exactly two dynamic arrays (`vm.rs:175–181`). The upstream fixtures `separate_length_field.psf`, `shadowsocks_padded.psf` and `padding.psf` use these features; the two shipped PSFs do not, and this item is therefore potential rather than demonstrated.

Affected: `third_party/proteus/src/lang/interpreter/vm.rs:173–199`, `:270`, `:294`, `:305`, `:361–392`; `third_party/proteus/src/lang/interpreter/io.rs:193–213`.

Proof of Concept: Static only. The `unchecked` subtraction at `vm.rs:368–371`:

```
ReadLength::IdentifierMinus((id, sub)) => {
   let num: &u128 = runtime.load(id)?;
   (*num as usize) - sub
}
```

Impact: An operator who adopts a padded or split-length PSF inherits a remote abort reachable through a crafted length field.

Preconditions: A PSF using padding or `length - N` expressions; none of the shipped PSFs.

Recommendation: Use `checked_sub` returning `lang::Error`, bound `read_exact` lengths by the format's declared maximum before allocating, replace the remaining `unwrap` and `assert!` sites with errors, and fuzz the server-role interpreter for every PSF that is deployed, under the release profile.

Classification: CWE-191 · OWASP ASVS 5.0.0 V15.3 (Defensive Coding) · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:N/VI:N/V

RVPN-01-215 WP2: The server starts with the shipped placeholder password in shadowsocks.psf (Low)

Fix Note: This issue has been fixed by the development team in commit 0df53c3 and verified by Red Team Ltd to be working as expected. Loading a PSF whose password is the shipped placeholder is now an error (`src/proteus_carrier.rs:65–78` at 32e743e, unit test `:579–590`); the release build of 9c4460a refused the unedited file with `Error: [PROTEUS] PSF shadowsocks-placeholder.psf still has the placeholder password REPLACE_WITH_PASSWORD; write your own into your copy` and exit status 1 (Appendix A, T129).

`Dispatch::load` reads and compiles the configured PSFs (`src/proteus_carrier.rs:165–173`) and nothing refuses a `@SEGMENT.CRYPTO` block whose password is the shipped placeholder `REPLACE_WITH_PASSWORD` (`proteus/shadowsocks.psf:24`); only a comment in the example configuration warns the operator (`config.example.toml:27–28`). A node started with the unedited file keys its outer layer with `SHA-256("REPLACE_WITH_PASSWORD")`, a value anyone can compute, and the first-frame authentication of RVPN-01-204 then accepts frames from anyone who read the repository.

Affected: `src/proteus_carrier.rs:165–173` (`Dispatch::load`), `:176–202`; `proteus/shadowsocks.psf:24`.

Proof of Concept: Server P was started with the unedited file (T80): `proteus psfs: tls_mimic.psf (prefix 166 bytes), shadowsocks-placeholder.psf (default), server P is running with the placeholder password.`

Impact: Combined with RVPN-01-206 the outer layer is then keyed with a public value; the Noise layer inside still protects the tunnel.

Preconditions: An operator who deploys the shipped file unchanged.

Recommendation: Refuse to start when a PSF's password equals the placeholder or is shorter than a minimum length, and move the password out of the PSF into the configuration file where other secrets live.

Classification: CWE-1188 · OWASP ASVS 5.0.0 V13.3 (Secret Management) · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:N

RVPN-01-216 WP2: PSF paths, per-connection routing and interpreter summaries are logged (Low)

Fix Note: This issue has been fixed by the development team in commits 575ceb1 and 32e743e. Red Team Ltd verified 575ceb1 against the release build of 9c4460a to be working as expected, and verified 32e743e by code review. Since 575ceb1 the start-up banner names each PSF by its file name (`src/proteus_carrier.rs:224-236` at 32e743e), and the MUX2 tunnel-open line, the routing line and the interpreter's completion line are logged at `debug` (`src/mux.rs:45-46`, `src/proteus_carrier.rs:378`, `third_party/proteus/src/lang/interpreter/mod.rs:190`); at `info`, the 9c4460a build logged no PSF path, routing line or interpreter summary during a v1 tunnel, a MUX2 tunnel and a standalone shadowsocks session. The one identity-bearing line it still logged at `info`, `[MUX2] tunnel closed ip=127.0.0.1 cid=0123456789abcdef age=0s bytes_up=36 bytes_down=207`, is emitted with `log::debug!` from commit 32e743e on (`src/mux.rs:177-184`; Appendix A, T133).

The start-up banner logs every PSF path at `info` (`src/main.rs:124` via `Dispatch::describe`, `src/proteus_carrier.rs:209-221`); each routing decision and every tunnel error with the peer address are logged at `debug` (`:336`, `:385-391`); the vendored interpreter logs one `info` line per connection with the byte counts in both directions (`third_party/proteus/src/lang/interpreter/mod.rs:190-201`); the MUX2 path logs `tz`, `app`, `os`, `cid`, `sni` and the client address at `info` (`src/mux.rs:45-53`). The default filter is `warn` (`src/main.rs:52`).

Affected: `src/main.rs:52`, `:122-126`; `src/proteus_carrier.rs:209-221`, `:336`, `:385-391`; `third_party/proteus/src/lang/interpreter/mod.rs:188-201`; `src/mux.rs:45-53`.

Proof of Concept: Server A's log at `debug` during T20 to T34: `[PROTEUS] 127.0.0.1:54681 -> /tmp/rvpn01/wp2/psf/tls_mimic.psf, [MUX2] tunnel open: tz=Asia/Tehran app=rostam os=android-14 cid=0123456789abcdef sni= ip=127.0.0.1`; the standalone client's log at `info`: `Interpreter done: AppToNet(130?306, Ok(306)), NetToApp(801?50, Ok(50))`.

Impact: A journal at `info` lists every connection and the carrier set of the node; at `debug` it maps every peer address to a PSF and records identities and destinations alongside addresses.

Preconditions: Log access, or an operator running at `info` or `debug`.

Recommendation: Log PSF base names rather than paths, demote the interpreter's completion line to `debug` in the vendored crate, and keep identities and addresses out of `info` lines.

Classification: CWE-532 · OWASP ASVS 5.0.0 V16.2 (General Logging) · CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/VA:N

RVPN-01-217 WP2: rustls 0.23.42 carries four advisories and rustls-pemfile is unmaintained (Low)

`cargo audit -n` against advisory database commit `edb58f59` reported four vulnerabilities in rustls 0.23.42 and one unmaintained crate (Appendix B): RUSTSEC-2026-0285 (TLS 1.3 handshake messages accepted across encryption-level boundaries, 5.3), RUSTSEC-2026-0346 (`Debug` implementations for client session values expose TLS secrets, 4.0), RUSTSEC-2026-0348 (Brotli certificate decompression can allocate up to 1 GiB before authentication, 3.7), RUSTSEC-2026-0349 (unauthenticated ServerHello can downgrade later connections to classical key exchange, 3.7), and RUSTSEC-2025-0134 (rustls-pemfile 2.2.0 unmaintained). rustls is the client side of the TLS connections to the statistics and messaging stores (`src/stats.rs:207-220`, `src/valkey.rs:145-176`).

Affected: `Cargo.lock` (rustls 0.23.42, rustls-pemfile 2.2.0); `Cargo.toml:43-45`.

Proof of Concept: Appendix A, B5.

Impact: The affected code paths are client-side TLS to operator-controlled stores with a pinned CA; exploitation requires a hostile or impersonated store. The fixes are available in rustls 0.23.46.

Preconditions: A compromised or impersonated Valkey endpoint.

Recommendation: Upgrade rustls to 0.23.46 or later and migrate PEM parsing to `rustls-pki-types`; re-vendor and re-lock.

Classification: CWE-1395 · OWASP ASVS 5.0.0 V15.2 (Security Architecture and Dependencies) · CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:L/VA:L/SC:N/SI:N/SA:N

RVPN-01-218 WP2: Test fixtures and comments contain publicly routable addresses outside the documentation ranges (Info)

Fix Note: This issue has been fixed by the development team in commits 97d8d55, 9c4460a, c6c9a2f and aa667b5 and verified by Red Team Ltd. After these commits, the CIDR comment and the egress-pool fixtures in `src/proxy.rs`, the metrics-bind fixture in `src/config.rs`, the bundle fixtures in `src/protection.rs` and the rules fixture in `src/messaging.rs` use documentation addresses; the last two were changed in commit 9c4460a. Kept by design in the policy tests of `src/proxy.rs` are the boundary values on either side of the reserved ranges and of the 2001:db8::/32 documentation range (lines 734-742 and 748-752 at 32e743e, including 172.32.0.1 as the first address outside 172.16.0.0/12) and the IPv6 unicast literals that the deny-list tests need as routable addresses (lines 743-746), together with the well-known public resolver addresses used as fixtures and for the IPv6 route probe (`src/proxy.rs:99`, `:671`, `:732–733`, `:747`, `:992`; `src/protection.rs:1635`, `:1641`, `:1815`). Commits c6c9a2f and aa667b5 moved the remaining dialer-ordering, UDP-header and rules fixtures in `src/proxy.rs` and `src/protection.rs` to documentation addresses, so the boundary values and resolver addresses listed above are the only non-documentation literals left, by design.

The redaction rules of this engagement require that public trees carry no live infrastructure addresses. The following locations contain IPv4 or IPv6 literals that are neither loopback, private, link-local, multicast nor within the RFC 5737 and RFC 3849 documentation ranges; the values are not reproduced here. Classes: routable IPv4 unicast addresses used as allow-list and egress-pool fixtures in `src/proxy.rs:731–751`, `:811–816`, `:903–911`, `:937–943`, `:959–966`; routable IPv6 unicast addresses in the same tests at `src/proxy.rs:744–750`, `:864–867`, `:941–942` ↪ ; routable IPv4 and IPv6 ranges and addresses as bundle fixtures in `src/protection.rs:1514–1528`, `:1805–1832`, `:2041–2056`; a routable IPv4 address as a metrics-bind fixture in `src/config.rs:553`; a routable IPv4 address in a rules fixture in `src/messaging.rs:1459`; and, in a code comment, a routable IPv4 address in `src/proxy.rs:148`. Two hostnames in `src/protection.rs:2009–2010` follow a CDN naming pattern but are short placeholders rather than distribution identifiers, and `src/protection.rs:2015` names the product's own domain.

Affected: `src/proxy.rs:148`, `:731–751`, `:811–816`, `:864–867`, `:903–911`, `:937–943`, `:959–966`; `src/protection.rs` ↪ `:1514–1528`, `:1805–1832`, `:2041–2056`; `src/config.rs:553`; `src/messaging.rs:1459`.

Proof of Concept: Static only; located with a pattern search over `src/` and filtered against the reserved and documentation ranges.

Impact: If any value is a fleet, infrastructure or client address it discloses that address in a public tree; if the values are arbitrary, the exposure is nil. Red Team Ltd could not determine which, and lists the locations for the maintainers to scrub.

Preconditions: None.

Recommendation: Replace the literals with RFC 5737 (`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`) and RFC 3849 (`2001:db8::/32`) addresses, choosing values outside the deny list where the test requires a routable address, and add a lint that rejects other literals.

Classification: CWE-200 · OWASP ASVS 5.0.0 V14.2 (General Data Protection) · CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/V

RVPN-01-219 WP2: The example configuration ships a usable keypair and the private key is accepted on the command line (Info)

Fix Note: This issue has been fixed by the development team in commit 03e58e1 and verified by Red Team Ltd to be working as expected. Both example files now ship `REPLACE_WITH_*` placeholders (`config.example.toml:4-5`, `client.example.toml:2` at 32e743e), and start-up refuses the placeholders and the published example key in `private_key`, in `extra_private_keys` and through `--private-key` (`src/config.rs:319-331`, `:351`, `:359`; `src/main.rs:67-71`); the release build of 9c4460a ended with exit status 1 in all four cases tested (Appendix A, T130). Commit c6c9a2f removes the `--private-key` option; the private key is read from the configuration file only and never appears in a process list.

`config.example.toml:3-4` carries a complete X25519 keypair and `client.example.toml:2` the matching public key, so an operator who copies the example without running `--keygen` deploys a key that is public; `--private-key` is accepted on the command line (`src/main.rs:37-39`), where it is visible to every local user through the process list. Both items exist in WP1.

Affected: `config.example.toml:3-4`; `client.example.toml:2`; `src/main.rs:37-39`; also affects WP1.

Proof of Concept: Static only; the keys are visible in the files named.

Impact: A node deployed with the example key is impersonable by anyone who read the repository; a key on the command line is readable by local users for the process's lifetime.

Preconditions: Operator error, or local access.

Recommendation: Ship `REPLACE_WITH_*` placeholders in both example files, refuse to start with the documented example key, and read the private key from the configuration file or a file descriptor only.

Classification: CWE-798 · OWASP ASVS 5.0.0 V13.3 (Secret Management) · CVSS:4.0/AV:L/AC:L/AT:P/PR:L/UI:N/VC:H/VI:N/

Conclusions

Red Team Ltd examined the redsocket_proteus exit server at commit `34f0050` as work package WP2 of engagement RVPN-01, combining a full source review of the server, its standalone client and the integration boundary of the vendored Proteus crate with a dynamic campaign of 128 recorded tests on the assessment host, including the re-verification of commits 97d8d55 and 9c4460a after the assessment window. The verdict is mixed in a specific way: the project's own code is small, disciplined and well tested, and the three issues inherited from the predecessor that the commit set out to fix are fixed; but the component the product now relies on for censorship resistance, the Proteus shaping layer with the two shipped specification files, is not fit for an Internet-facing deployment in its current form, and one of its defects is severe enough to take a node, and by repetition a fleet, offline with eighteen bytes.

The carrier and dispatch logic held up well. Every PSF is compiled once at start-up, a file that does not parse, does not compile or does not exist stops the server naming the file, and the parser's panics are converted to errors on that path (`src/proteus_carrier.rs:43-57`, tested `:527-534`); a configuration without a PSF is refused (`src/config.rs:350-355`). The dispatch rules are explicit and tested: the longest fixed prefix wins, at most one PSF without a fixed prefix is allowed, silent peers are closed, and an HTTP request line is claimed by no PSF (`:105-153`, `:434-534`). Nothing is answered in clear, which was verified against the debug and the release build, and the legacy `[http]` table is ignored as documented. Memory shedding runs before the PROXY peek, the first-bytes peek and any allocation and refuses silently, which is indistinguishable from an unmatched connection; it could not be exercised on macOS, where the shedder correctly reports that it is blind. The listener uses `SO_REUSEADDR` and `SO_REUSEPORT` for zero-downtime restarts, which the testing team observed when a second instance bound an in-use port successfully (T83); this is a documented feature and is restricted by the kernel to the same user, but operators should know that a port is not exclusive.

The interpreter's lifetime is bounded as designed. A bare ClientHello, a stalled oversized record and a stalled Noise length prefix were each closed at 600.01 s (L1, L3, L6), and an idle established relay, whose detached tasks outlive the budget, was closed at 630.0 s when the carrier aborted the interpreter after the 30 s grace (L2; server log: `Connection timeout (600s)` and `interpreter ... did not unwind in time`). This resolves, for this server, the predecessor's finding that relays outlived the cap. Per-connection memory is bounded by the 64

KiB duplex and the u16 length fields of both PSFs; malformed yamux frames end only the tunnel that sent them (T32, T33), and the 256-stream limit is enforced (T34). The resilience gaps the assessment found, an accept loop that ended the process on a descriptor-exhaustion error and a silently ignored extra bind (RVPN-01-210, T84), were closed in commit 9175f01: the release build of 9c4460a waited out descriptor exhaustion and kept serving, and an extra bind on a held port stopped start-up (T131, T132).

The relay and egress policy held. Fourteen address forms of the loopback origin, including IPv4-mapped IPv6, the unspecified addresses, link-local, carrier-grade NAT, multicast, the documentation range and three non-canonical spellings resolved through the system resolver, were refused on the server with the default deny list (T75), as were the MUX2 and UDP paths (T75_mux, T78), while the same requests succeeded on the control server with the list disabled (T76, T77); the refusal counter advanced accordingly. Control calls bypass the policy by design; since commit 534ecf6 they return only the session's own cid and the Noise public key rather than node-wide figures (RVPN-01-209, T128).

Cryptography splits cleanly into the layer that is sound and the layer that is not. The Noise NK session is unchanged from WP1: `Noise_NK_25519_ChaChaPoly_BLAKE2s` via snow with u16 framing, a 48-byte message one and a 48-byte message two in every observed handshake, a fresh responder per rotation key, forward secrecy from the ephemeral exchange, and immediate, uniform closure on a malformed, truncated, zero-length or wrong-key message one and on a corrupted first transport message (T24 to T28). The Noise layer is also what limits the damage of everything above it: whatever the PSF does, the tunnel content is encrypted and the server is authenticated inside. The shaping layer is where the assessment found its substantive problems. `tls_mimic.psf` is a public fixture whose handshake violates the TLS state machine, repeats a 2013 timestamp and an expired 1024-bit SHA-1 test certificate, selects a compression method the client did not offer, carries no SNI, and answers its 166 published bytes with 746 deterministic bytes (RVPN-01-205). `shadowsocks.psf` keys every connection with `SHA-256(password)`, restarts a fixed nonce sequence per connection, and therefore produces identical first eighteen bytes in each direction on every connection from every client to every node sharing the file, reuses nonces across connections, and answers a replayed first frame with its constant opener on any node (RVPN-01-206, demonstrated on two instances). Neither property can be repaired by configuration; both need changes in the vendored crate or a different carrier. The vendored crate's README states that "Proteus is experimental and not recommended for security-critical applications" (`third_party/proteus/README.md:16-17`), and the project's README repeats the caution (`README.md:219-227`); the findings of this report are the concrete form of that caution.

The most serious finding sits at the junction of the two layers. With the example configuration, the shadowsocks interpreter authenticates the first eighteen bytes of every connection that is not the fixed ClientHello, and the vendored interpreter unwrapped the result; under the release profile's `panic = "abort"` the process terminated (RVPN-01-204). Red Team Ltd reproduced this with a clear-text GET, with eighteen random bytes, with a partial ClientHello that stalled, and with a PROXY line under the default mode, each time with exit status 134 and the panic location in the log, and confirmed that a `tls_mimic`-only configuration is not affected. Every piece of evidence in the repository that garbage "is closed at once" was produced under the debug profile, where a panic ends only the task; the shipped profile was never exercised by the project's tests or scripts. The fix is a few lines in the vendored crate, and the test that would have caught it is a release-profile run of the existing end-to-end script. Commit 97d8d55, pushed after the assessment window, replaces the unwraps with errors and adds checked length arithmetic and a 16 MiB read cap; Red Team Ltd re-ran the proofs of concept against its release build and found every such input closed with an interpreter error while the process kept serving tunnels (Appendix A, T110 to T117).

Configuration and operational surface improved. The `proxy_protocol` default is `disabled` by enum default and is pinned at the `Config` level (RVPN-01-202), the Valkey password is redacted from the banner (RVPN-01-203), and the PROXY parser remains length-bounded and tested over real sockets. The items that remained at the commit under review were fixed in the remediation pass and re-verified against the release build of 9c4460a: the PROXY stage closes silent and stalled peers at 5 s, and `auto` strips a header without trusting its address while `required` records it (RVPN-01-208, T120 to T124); LOCAL, UNSPEC and UNKNOWN headers record the real socket peer (RVPN-01-212, T125 to T127); and start-up refuses the placeholder password (RVPN-01-215, T129) as well as the published example key and the key placeholders (RVPN-01-219, T130). The `--private-key` override remains, and a key passed through it is visible in the process list.

Identity remains the unresolved design question carried over from WP1. The server has no client identity beyond

the asserted cid, and the dynamic campaign showed, against a live store, that any party holding a cid reads that client's notices and entitlement, rewrites its rules, switches off its protection categories and silences its messages, and that usage rows carry whatever cid a client claims (RVPN-01-207). The messaging module is off by default and should stay off on public exits until the protocol carries a verifiable identity.

Data handling is bounded but identity-bearing: statistics rows bind the client address, cid, time zone, application, operating system and SNI. Since commits 575ceb1 and 32e743e the logs no longer repeat this at `info`: the MUX2 lines, the routing decisions and the interpreter's per-connection summaries are logged at `debug`, and the banner names PSFs by file name rather than by path (RVPN-01-216, T133). The standalone client binds its SOCKS5 and UDP sockets to loopback only, pins the server's public key, requires a PSF and compiles it behind `catch_unwind`; it shares the interpreter's unwrap and the release profile, so an impersonated or mis-keyed exit speaking `shadowsocks.psf` terminates it with the first eighteen bytes.

Supply chain and dependencies: the tree builds offline from vendored sources, `cargo clippy` is clean, the test suite passes, and the Proteus dependency tree added 26 crates with no advisory of their own; rustls needs an upgrade (RVPN-01-217). No SBOM document (CycloneDX or SPDX) and no build provenance (SLSA) accompany the tree, and the vendored crate's correspondence to its upstream commit could not be verified offline (Appendix B).

Prioritised recommendations. First, before any Internet-facing deployment of this server, propagate errors instead of unwrapping in `third_party/proteus/src/lang/interpreter/vm.rs` and run the end-to-end script against a release build; until then deploy without a default PSF that authenticates its first bytes. Second, retire both shipped PSFs as production carriers: replace `tls_mimic.psf` with a real TLS termination or a faithful, randomised imitation, and give the shadowsocks-shaped carrier a per-connection salt, a derived key and counter nonces, or move probe resistance into the carrier with a first-frame authenticator derived from the server's static key. Third, decide the client-identity protocol before `[messaging]` is enabled on a public node, and treat usage attribution as unauthenticated until then. Fourth, keep under test the remediations of the accept loop, the extra binds, the PROXY stage and the start-up refusals of the placeholder password and the example key (RVPN-01-208, -210, -215, -219), and retire the `--private-key` override in favour of the configuration file. Fifth, upgrade rustls, scrub the non-documentation addresses from the fixtures, publish an SBOM with releases, and record the vendored crate's upstream commit and edits in a machine-readable form. Field tests that this report could not perform and that would sharpen the fingerprinting findings are: a capture of one genuine session of each PSF run through a TLS-aware classifier such as Zeek or Suricata with default rules, and a comparison of the silent-peer, reset and half-close timings against a reference nginx on port 443.

Red Team Ltd thanks the RostamVPN maintainers for a clean, well-documented and fully vendored tree, for the honest statement of the shaping layer's experimental status in the README, and for the three remediations that were already in the commit under review; the quality of the integration code made this assessment considerably easier than its findings might suggest.

Appendix A: Dynamic Test Log

Environment: assessment host as described in Test Methodology and Coverage; all listeners on 127.0.0.1 within ports 20200 to 20299; scratch directory `/tmp/rvpn01/wp2/`. Server instances and ports: A 20201 (debug, both PSFs, `deny_cidrs = []`), B 20203 (debug, both PSFs, default deny list), C 20205 (release, both PSFs), D 20207 (debug, `tls_mimic.psf` only), E 20209 (`auto`), F 20211 (`required`), G 20215 (`[messaging]` and `[stats.valkey]` against redis-server on 20213 plain and 20214 TLS), P 20217 (placeholder password), tls_mimic-only release 20227; origin HTTP server 20221; silent TCP sink 20222; UDP echo 20223; logging relays 20240 and 20241; standalone clients on 20230 to 20236. `raw.sh` is a byte-level TCP probe (steps: `get`, `upgrade`, `hello`, `hello_partial:N`, `garbage:N`, `record:HDR:LEN:FILL:COUNT`, `proxy_v1:LINE`, `proxy_v2:CMD:FAM:BODY`, `hex:`, `file:`, `silent`, `proxy_stall`); `rvpnprobe` is the purpose-written Rust probe (`hello`, `noise`, `mux` subcommands). Test keys were generated with `--keygen` for the campaign and are elided as `<pubkey>`; the assessment host's name in statistics rows is masked as `<host>`; paths inside quoted panic messages are shown relative to the repository root. Outputs are trimmed to the lines that carry the result. The three-second and eight-second reads are the probe's observation windows, not server timers.

Build and tooling

- B1 `cargo build --offline --bins` (debug): `Finished dev profile`, exit 0; `redsocket-http` and `client` produced.
- B2 `cargo build --offline --release --bins`: `Finished release profile [optimized]`, exit 0.
- B3 `cargo test --offline`: 94 passed, 0 failed, 1 ignored over twelve result sets (unit tests, six e2e binaries, the harness and documentation tests); exit 0.
- B4 `cargo clippy --offline --all-targets`: `Finished dev profile`, no warnings.
- B5 `cargo audit -n` (database commit `edb58f59`, 1,329 advisories, 287 crates): RUSTSEC-2026-0346, RUSTSEC-2026-0285, RUSTSEC-2026-0348, RUSTSEC-2026-0349 (rustls 0.23.42), RUSTSEC-2025-0134 (rustls-pemfile unmaintained); `error: 4 vulnerabilities found!`, exit 1.
- C-1 `python3 -I` decode of `proteus/tls_mimic.psf` and `openssl x509 -inform DER -noout -subject -issuer -dates -serial -text` on the extracted certificate: results as listed in RVPN-01-205 (JA3 `d092a941ea714ee8a125b386f788d07a`; `CN=Test Certificate RSA`; `notBefore=Sep 15 21:51:10 2013 GMT`, `notAfter=Sep 13 21:51:10 2023 GMT`; `Public-Key: (1024 bit)`; `sha1WithRSAEncryption`; ServerHello compression `01`; no `server_name` extension).

Byte-level probes against servers A and D (debug)

- T01 `raw.sh --port 20201 --steps get --read 3`: 37 bytes sent; `t+0.01s connection reset/error: Connection reset by peer`. No HTTP response.
- T02 `raw.sh --port 20201 --steps upgrade --read 3`: 83 bytes sent; reset at t+0.01 s. No 101.
- T03 `raw.sh --port 20201 --steps hello --read 3`: 166 bytes sent; `first data: 746 bytes`, `ServerHello match : yes`; still open after 3 s.
- T04 `raw.sh --port 20201 --steps garbage:70000 --read 3`: reset at t+0.00 s.
- T05 `raw.sh --port 20201 --steps silent --read 8`: `t+5.01s peer closed the connection (EOF)after 0 bytes received`.
- T06 `raw.sh --port 20201 --steps hello_partial:100 --read 8`: `t+5.00s connection reset/error: Connection reset by peer` (partial prefix handed to the default PSF at the deadline).
- T07 `raw.sh --port 20207 --steps hello_partial:100 --read 8` (D, no default): reset at t+5.00 s.
- T08 `raw.sh --port 20207 --steps get --read 3` (D): reset at t+0.00 s.
- T09 `raw.sh --port 20207 --steps hello --read 3` (D): 746 bytes, match yes.
- T10 `raw.sh --port 20201 --steps hello,record:170303:50:zero:50 --read 35`: 746 bytes, then `peer closed the connection (EOF)` at t+0.00 s (zero-length Noise frame rejected, FIN immediately).
- T11 `raw.sh --port 20201 --steps hello,record:160303:50:zero:50 --read 35`: identical to T10; the `16 03 03` header was accepted as a record header.
- T12 `raw.sh --port 20201 --steps hello,hex:1703 --read 8`: 746 bytes; still open and silent after 8 s (truncated record header held).
- T13 `raw.sh --port 20201 --steps hello,record:170303:65535:random:300 --read 8`: 746 bytes; still open after 8 s (declared 65535-byte record held until the cap, see L3).
- T14 `raw.sh --port 20201 --steps garbage:18 --read 3`: `peer closed the connection (EOF)after 0 bytes received` at t+0.00 s.
- T15 `raw.sh --port 20201 --steps garbage:17 --read 8`: still open and silent after 8 s (interpreter waiting for the 18th byte).

Noise and MUX2 probes against server A

- T20 `rvpnprobe noise --server 127.0.0.1:20201 --pubkey <pubkey> --mode valid --connect 127.0.0.1:20221 --atyp ip4 --read 3`: ClientHello answered; `Noise msg1 sent (48 bytes)`; `Noise msg2 received (48 bytes)`; META and CONNECT sent; origin's `HTTP/1.0 200 OK` and body `wp2-origin-ok-...` received in plaintext; EOF after the response.
- T21 same with `--atyp domain`: identical result.
- T22 `... --mode control --connect _rs.stat:0`: plaintext `{"cid":"0123456789abcdef","active_tunnels":1,"total_tunnels":3,"bytes_up":72,"bytes_down":414}`.
- T23 `... --mode control --connect _rs.config:0`: plaintext `{"noise_pubkey":"<pubkey>","carriers":["proteus"],"server":"redsocket-http"}`.
- T24 `... --mode len0 --idle 35`: Noise frame of length 0 sent; `peer closed the connection (EOF)` at t+0.00 s.
- T25 `... --mode random48 --idle 35`: EOF at t+0.00 s.

- T26 ... `--mode wrongkey --idle 35`: EOF at t+0.00 s.
- T27 ... `--mode truncated --idle 8`: length 48 with 10 bytes then stall; still open after 8 s.
- T28 ... `--mode badct --idle 35`: handshake complete, 50 random bytes as first transport message; EOF at t+0.00 s.
- T29 ... `--mode valid --header 000000 --connect 127.0.0.1:20221 --atyp ip4`: handshake, CONNECT and the origin's response all succeeded with every client record carrying header `00 00 00`.
- T30 `rvpnprobe mux --server 127.0.0.1:20201 --pubkey <pubkey> --psf tls --mode connect --connect 127.0.0.1:20221 --atyp ip4 --wait 3`: stream reply `0500000100000000000`; 207 bytes from the origin.
- T31 same with `--psf ss`: identical result over the shadowsocks PSF.
- T32 ... `--mode badversion`: raw yamux frame with version 9 after the MUX2 header; 30 encrypted bytes back (yamux ping), then the tunnel ended; server log `yamux accept error: decode error: unknown version: 9`; process alive.
- T33 ... `--mode hugelen`: data frame declaring 4294967295 bytes; server log `frame body is too large (4294967295)`; tunnel closed; process alive.
- T34 ... `--mode flood --count 300`: `256 streams opened without a request`; `stream #257 refused: open timed out`.

PROXY protocol handling (servers E auto, F required, A disabled)

- T40 `raw.sh --port 20209 --steps "proxy_v1:PROXY TCP4 198.51.100.7 203.0.113.1 51234 443,hello"`: 746 bytes; server E log `198.51.100.7:51234 -> tls_mimic.psf`.
- T41 `raw.sh --port 20209 --steps "proxy_v2:1:0x11:c6336408cb007101109201bb,hello"`: 746 bytes; log `198.51.100.8:4242 -> tls_mimic.psf`.
- T42 `raw.sh --port 20209 --steps "proxy_v2:0:0x00:,hello"` (LOCAL): 746 bytes; log `0.0.0.0:0 -> tls_mimic.psf`.
- T43 `raw.sh --port 20209 --steps "proxy_v2:1:0x00:,hello"` (AF_UNSPEC): 746 bytes; log `0.0.0.0:0 -> tls_mimic.psf`.
- T44 `raw.sh --port 20209 --steps "proxy_v1:PROXY UNKNOWN,hello"`: 746 bytes; log `0.0.0.0:0 -> tls_mimic.psf`.
- T45 v2 header with version nibble 3 then hello: reset; log `PROXY v2 wrong version`.
- T46 v1 line of 206 bytes without CRLF then hello: reset; log `PROXY v1 too long`.
- T47 v2 AF_INET header with 20 bytes of TLVs then hello: 746 bytes; log `198.51.100.8:4242 -> tls_mimic.psf`.
- T48 `raw.sh --port 20209 --steps hello` (auto, no header): 746 bytes; log `127.0.0.1:... -> tls_mimic.psf`.
- T49 `raw.sh --port 20211 --steps hello` (required, no header): reset; log `PROXY protocol header required but not present`.
- T50 `raw.sh --port 20211 --steps "proxy_v1:PROXY TCP6 2001:db8::7 2001:db8::1 51234 443,hello"`: 746 bytes; log `[2001:db8::7]:51234 -> tls_mimic.psf`.
- T51 `raw.sh --port 20201 --steps "proxy_v1:PROXY TCP4 198.51.100.7 203.0.113.1 51234 443,hello"` (disabled): reset at t+0.00 s (bytes routed to the default PSF).
- T52, a log search with a mis-quoted pattern, produced no output and was replaced by the server-E and server-F log excerpts quoted under T40 to T51.

Captures and replays through the logging relay (target server A)

- T60 standalone client 1 (`shadowsocks.psf`, cid `a0a0...`) via SOCKS5 on 20232 through relay 20240: `curl --socks5-hostname 127.0.0.1:20232 http://127.0.0.1:20221/hello.txt` returned the origin file.
- T61 standalone client 2 (`shadowsocks.psf`, cid `b1b1...`) on 20233: same result.
- T62 comparison of the captured first bytes: `conn1.c2s` and `conn2.c2s` 337 bytes each, first 18 bytes identical (`003b78f5060241fec6c9eb9f53649e6e641d`), bytes 18 onward different; `conn1.s2c` and `conn2.s2c` first 18 bytes identical (`eade12bbdfbe44894ad745815579445e9dde`).
- T63 `raw.sh --port 20201 --steps file:cap/ss/conn1.c2s --read 3`: 337 bytes replayed; `first data: 84 bytes: eade12bb...`; EOF after 84 bytes.
- T63b the replay reply's first 18 bytes equal the captured server opener of connection 1.
- T64 standalone client 3 (`tls_mimic.psf`) on 20234 through relay 20241: origin file returned.
- T65 captured tls_mimic bytes: client 166-byte ClientHello followed by `17 03 03 00 32 | 00 30 | 48-byte msg1` and a `17 03 03 00 50` record.

- T66 `raw.sh --port 20201 --steps file:cap/tls/conn1.c2s --read 3`: 400 bytes replayed; 746 bytes then a 55-byte record (fresh msg2); 801 bytes total; then EOF.
- T79 `raw.sh --port 20203 --steps file:cap/ss/conn1.c2s --read 2` (server B): `first data: 84 bytes: eade12bb...`.
- T79b openers compared: A capture, A replay and B replay all begin `eade12bbdfbe44894ad745815579445e9dde`.

Messaging and statistics (server G, redis-server 8.4.0 with `--requirepass`, TLS on 20214 with a scratch CA)

- T70 `rvpnprobe mux --server 127.0.0.1:20215 --pubkey <pubkey> --meta "MUX2|Asia/Dubai|rostam|android-14|a1a2a3a4a5a6a7a8||msg1" --mode control --wait 4`: control stream opened (magic `0xc0`); downstream `protection` status, the two seeded `notice` entries of that cid, and its `entitlement` (`state: paid, tier: pro, paid_until: 2027-01-01T00:00:00`).
- T71 same with `--rules '{"rules":{"allow":["other.example"],"deny":[],"categories_off":65535}}' --ack 2`: downstream echo `{"seq":4,...,"type":"rules","payload":{"allow":["other.example"],"categories_off":65535,"deny":[],...}}`.
- T72 `redis-cli HGETALL rules:a1a2a3a4a5a6a7a8`: `allow ["other.example"]`, `deny []`, `categories_off 65535`; `GET msg:a1a2a3a4a5a6a7a8:last` = 2.
- T73 a second client with cid `ffffffffffffffff` relayed one request through MUX2 (origin response received); the TLS read of the statistics lists in this step failed on a relative path and was repeated as T73r.
- T73r `redis-cli -p 20214 --tls --cacert ca.pem LRANGE stats:cid 0 -1` and `LRANGE stats:tunnel-closed 0 -1`: three presence rows and three usage rows carrying the asserted cids `a1a2a3a4a5a6a7a8` (twice) and `ffffffffffffffff`, with the client address `127.0.0.1`, `Asia/Dubai`, `rostam`, `android-14` and the host name.

Destination policy (server B default deny list; server A control)

- T75_1 to T75_14 `rvpnprobe noise --server 127.0.0.1:20203 --pubkey <pubkey> --mode valid --connect <addr>:20221 --atyp <type>` for `127.0.0.1` (ip4), `localhost` (domain), `::1` (ip6), `::ffff:127.0.0.1` (ip6), `0.0.0.0` (ip4), `169.254.1.1` (ip4), `10.0.0.1` (ip4), `100.64.0.1` (ip4), `192.0.2.1` (ip4), `224.0.0.1` (ip4), `127.1` (domain), `2130706433` (domain), `0x7f000001` (domain), `::` (ip6): every CONNECT was followed by `peer closed the connection (EOF)` without data; server log `refused unroutable destination ... none routable from an exit` for each; counter `redsocket_egress_refused_unroutable` advanced from 1 to 16 over the batch.
- T75_mux `rvpnprobe mux ... --mode connect --connect 127.0.0.1:20221` (B): `stream ended before a reply: early eof`; log `target connect 127.0.0.1:20221 failed: refused unroutable destination`.
- T75_ctl `rvpnprobe noise ... --mode control --connect _rs.config:0` (B): banner returned; control calls are not subject to the policy.
- T76 `rvpnprobe noise --server 127.0.0.1:20201 ... --connect 127.0.0.1:20221 --atyp ip4` (A): origin response received.
- T77 `socks_udp.py 20235 127.0.0.1 20223` through the standalone client (MUX2) to server A: `UDP ASSOCIATE reply: 0500000...`; `datagram back: b'echo:ping-udp'`.
- T78 `socks_udp.py 20236 127.0.0.1 20223` through the standalone client to server B: association accepted; `no datagram back within 3 s`.

Configuration-level tests

- T80 server P with the unedited `shadowsocks.psf`: started; banner names the placeholder file as the default PSF; process running.
- T81 configuration with `[http] proxy_protocol = "auto"` and no key under `[proteus]`: banner `proxy_protocol : Disabled`.
- T82 `[protection] enabled = true` with a `[protection.valkey]` table: banner renders `password: "<redacted>"`; the password string does not appear in the log.
- T83 `extra_binds = ["127.0.0.1:20201"]` while server A held the port: both instances listed `Listening on 127.0.0.1:20201` (`SO_REUSEPORT`); not a failure case, superseded by T83r.
- T83r `extra_binds = ["127.0.0.1:20221"]` (held by the origin without `SO_REUSEPORT`): `extra bind 127.0.0.1:20221 failed: Address already in use (os error 48)`; server still running on its primary bind and serving the ClientHello.
- T84 server S under `ulimit -n 40`, 60 idle connections attempted: `opened 30 idle connections`; `server S has exited`; log `Error: Too many open files (os error 24)`.

Release-profile behaviour (server C and a tls_mimic-only release instance)

- T90 `raw.sh --port 20205 --steps hello`: 746 bytes, match yes.
- T91 `rvpnprobe noise --server 127.0.0.1:20205 ... --mode valid --connect 127.0.0.1:20221 --atyp ip4`: full tunnel, origin response received.
- T92 `raw.sh --port 20205 --steps get`: reset at t+0.00 s; server log `panicked at third_party/proteus/src/lang/interpreter/vm.rs:267:54: called Result::unwrap()on an Err value: Crypto(Decrypt(Decrypt(Error ))))`. The process-state check attached to T92 to T96 (`T92b` to `T96b`) referenced an unexported variable and was invalid; it was superseded by T92r and T96r.
- T92r fresh release instance in the foreground; `raw.sh --port 20205 --steps get --read 2`; `wait`: `Abort trap : 6`, `server exit status: 134`, panic line as in T92.
- T93 restarted instance; `raw.sh --port 20205 --steps garbage:18`: EOF at t+0.00 s; panic line in the log; instance gone.
- T94 restarted instance; `raw.sh --port 20205 --steps hello_partial:100 --read 8`: reset at t+5.00 s; panic line in the log.
- T95 restarted instance; `raw.sh --port 20205 --steps "proxy_v1:PROXY TCP4 198.51.100.7 203.0.113.1 51234 443,hello"`: reset; panic line in the log.
- T96 restarted instance; `raw.sh --port 20205 --steps hello,record:170303:50:zero:50`: 746 bytes then EOF (no cipher on this path).
- T96r fresh release instance; same input as T96; `ps` shows the process alive afterwards (`S`), exit status 143 only when killed by the testing team; then a tls_mimic-only release instance on 20227 received 18 random bytes and a GET (both reset) and was alive afterwards (`S`), exit status 143 on kill.
- T97 `raw.sh --port 20227 --steps get` (tls_mimic-only release): reset; process alive (T96r).
- T98 `raw.sh --port 20227 --steps garbage:18`: reset; process alive (T96r).

Long-running tests (server A unless stated)

- L1 `rvpnprobe hello --server 127.0.0.1:20201 --idle 720`: ClientHello answered; `[t+600.01s] peer closed the connection (EOF).`
- L2 standalone client on 20230 (tls_mimic, single stream) to server A; `socks_idle.py --socks 20230 --target 127.0.0.1:20222 --wait 720` (silent sink): SOCKS5 CONNECT accepted at t+0.0 s; `t+630.0s EOF from the client-side SOCKS connection (tunnel ended)`; server log `Connection timeout (600s)` followed by `interpreter ... did not unwind in time.`
- L3 `raw.sh --port 20201 --steps hello,record:170303:65535:zero:1000 --read 720`: 746 bytes; `t+600.01s peer closed the connection (EOF).`
- L4 `raw.sh --port 20209 --steps proxy_stall --read 90` (E, auto): `PROXY` sent; `t+90.01s still open and silent after 90.0 s.`
- L6 `rvpnprobe noise --server 127.0.0.1:20201 ... --mode len65535 --idle 720`: `[t+600.01s] peer closed the connection (EOF).`
- L7 `raw.sh --port 20209 --steps silent --read 90` (E, auto): still open after 90 s.
- L8 `raw.sh --port 20211 --steps silent --read 90` (F, required): still open after 90 s.

Re-verification against commit 97d8d55 (after the assessment window; release and debug binaries and the probe rebuilt from that commit; Fix Notes of RVPN-01-204, -214 and -218)

- T110 release build with the example PSF pairing, started with its PID recorded; `raw.sh --port 20205 --steps get --read 2`: `peer closed the connection (EOF)after 0 bytes received`; process state `S` afterwards; `panic lines in log: 0`; log `interpreter for ... ended: Error in net to app direction: An interpreter crypto error occurred: Decrypt(Decrypt(Error))`.
- T111 same instance, `raw.sh --port 20205 --steps garbage:18`: EOF after 0 bytes; process alive.
- T112 same instance, `raw.sh --port 20205 --steps hello_partial:100 --read 8`: reset at t+5.00 s; process alive.
- T113 same instance, PROXY v1 line then ClientHello under `disabled`: reset; process alive.
- T114 same instance, `garbage:70000` and then `rvpnprobe noise ... --mode valid --connect 127.0.0.1:20221 --atyp ip4`: origin response received in plaintext; process alive; `panic lines in log: 0`.
- T115 standalone client with `shadowsocks.psf` against the same release build: `curl --socks5-hostname` returned the origin file (regression check).
- T116 debug build, `raw.sh --port 20201 --steps get`: reset; log `interpreter for ... ended: Error in net to`

↪ app direction: An interpreter crypto error occurred: Decrypt(Decrypt(Error)); no panic line.

- T117 scratch program linking the crate at 97d8d55: the server-role interpreter of the shipped shadowsocks ↪ .psf was fed authenticated length fields of 0, 5 and 15 (18-byte prefixes produced with the crate's own cipher under the test password) and the server-role interpreter of the upstream padding.psf fixture was fed six random 64-byte frames; under the unwind and the abort strategy every decoded underflow ended with Error in net to app direction: wire length is smaller than the bytes already consumed, the task ended without a panic, the peer saw a reset and the program completed; frames whose random lengths were valid remained waiting for more bytes.

Re-verification against commit 9c4460a (after the maintainers' remediation pass; release and debug binaries and the probe rebuilt from that commit; every server below is the release build, started with its PID recorded: A2 on 20201 with both PSFs, E2 on 20209 with proxy_protocol = "auto", F2 on 20211 with proxy_protocol = " ↪ required", S on 20225; Fix Notes of RVPN-01-208, -209, -210, -212, -215, -216 and -219)

- T120 raw.sh --port 20209 --steps silent --read 8 (E2, auto): t+5.00s peer closed the connection (EOF) ↪ after 0 bytes received (L7 at 34f0050: still open after 90 s).
- T121 raw.sh --port 20209 --steps proxy_stall --read 8 (E2, auto): PROXY sent (6 bytes); t+5.00s peer ↪ closed the connection (EOF)after 0 bytes received (L4: still open after 90 s).
- T122 raw.sh --port 20211 --steps silent --read 8 (F2, required): t+5.00s peer closed the connection (EOF ↪)after 0 bytes received (L8: still open after 90 s).
- T123 raw.sh --port 20209 --steps "proxy_v1:PROXY TCP4 198.51.100.7 203.0.113.1 51234 443,hello" -- ↪ read 2 (E2, auto): 746 bytes; server E2 log 127.0.0.1:65485 -> tls_mimic.psf, the socket peer (T40 at 34f0050 recorded the header's address).
- T124 the same header to F2 (required) on 20211: 746 bytes; log 198.51.100.7:51234 -> tls_mimic.psf.
- T125 raw.sh --port 20211 --steps "proxy_v2:0:0x00:,hello" --read 2 (F2, LOCAL): 746 bytes; log 127.0.0.1:65487 -> tls_mimic.psf (T42: 0.0.0.0:0).
- T126 raw.sh --port 20211 --steps "proxy_v2:1:0x00:,hello" --read 2 (F2, AF_UNSPEC): 746 bytes; log 127.0.0.1:65488 -> tls_mimic.psf (T43: 0.0.0.0:0).
- T127 raw.sh --port 20211 --steps "proxy_v1:PROXY UNKNOWN,hello" --read 2 (F2): 746 bytes; log 127.0.0.1:65489 -> tls_mimic.psf (T44: 0.0.0.0:0).
- T128 rvpnprobe noise --server 127.0.0.1:20201 --pubkey <pubkey> --mode control --connect _rs.stat:0 ↪ --read 2, then the same with --connect _rs.config:0 (A2): plaintext {"cid":"0123456789abcdef"} and {"noise_pubkey":"<pubkey>"}, 30 and 87 bytes including the length prefix (T22 and T23: 98 and 136 bytes with counters, carrier list and server name).
- T129 redsocket-http --config P.toml (the unedited shadowsocks.psf of T80): Error: [PROTEUS] PSF ↪ shadowsocks-placeholder.psf still has the placeholder password REPLACE_WITH_PASSWORD; write your ↪ own into your copy; exit status: 1.
- T130 start-up with K1 (private_key set to the published example key), K2 (the REPLACE_WITH_* placeholders), K3 (the campaign key as private_key and the published key in extra_private_keys) and configuration A with --private-key set to the published key: Error: private_key is the published example key; generate ↪ a keypair with --keygen (K1 and --private-key), Error: private_key is still the example placeholder ↪ ; generate a keypair with --keygen (K2), Error: extra_private_keys[0] is the published example key; ↪ generate a keypair with --keygen (K3); exit status 1 in all four cases.
- T131 server S under ulimit -n 40; 60 idle connections opened, held for 3 s and closed; then raw ↪ .sh --port 20225 --steps hello --read 1: opened 60 idle connections; accept retry lines: 4; log [ACCEPT] 127.0.0.1:20225: Too many open files (os error 24); retrying in 1s; server S still running; ClientHello answered with 746 bytes (T84 at 34f0050: process exited).
- T132 configuration R3 with extra_binds = ["127.0.0.1:20221"], the port held by the origin: Error: cannot ↪ bind 127.0.0.1:20221: Address already in use (os error 48); exit status: 1; listeners left on the ↪ primary bind: 0 (T83r: kept running on its primary bind).
- T133 A2 at info: banner proteus psfs: tls_mimic.psf (prefix 166 bytes), shadowsocks.psf (default) ↪ ; a v1 tunnel (rvpnprobe noise ... --mode valid --connect 127.0.0.1:20221 --atyp ip4, 2 plaintext frames), a MUX2 tunnel (rvpnprobe mux ... --mode connect, 1 response) and a standalone client session over shadowsocks.psf on 20238 (curl --socks5-hostname returned wp2-origin-ok-3275514607); log lines at info containing cid=, ip=, a PSF path or Interpreter done: one, [MUX2] tunnel closed ip=127.0.0.1 cid ↪ =0123456789abcdef age=0s bytes_up=36 bytes_down=207; all [MUX2], [NoiseProxy] and [PROTEUS] lines at

`info`: that line and `[PROTEUS] Listening on 127.0.0.1:20201`. The E2 and F2 logs show the routing and interpreter lines at `debug`, the routing lines naming the PSF by file name only.

- T120 to T122 and T128 ran against the first start of A2, E2 and F2, whose batch script passed an empty `RUST_LOG` instead of the intended level; T123 to T127 and T133, which read the server log, found no log lines in that run and were repeated after the three servers had been restarted with A2 at `info` and E2 and F2 at `debug`. The entries above record the repeated runs.

Appendix B: Dependency Inventory and Advisories

`Cargo.lock` resolves 287 crates, all vendored under `vendor/` and resolved offline through `.cargo/config.toml`; the build is reproducible from the tree without network access, and `build.rs` embeds the short commit and the build date in the version string (banner `RedSocket Proteus v3.5.0 (34f0050 ...)`). Relative to the WP1 lock (263 crates), `io-uring 0.6.4` and `sha1 0.10.7` were removed and the Proteus dependency tree added 26 crate versions: asn1 0.23.1, asn1_derive 0.23.1, enum-from 0.1.0 (path, `third_party/proteus/lib/enum-from`), fast-socks5 1.0.0, fixed-bitset 0.5.7, foldhash 0.1.5, hashbrown 0.15.5 (second version), itertools 0.14.0 (second version), pem 3.0.6, pest 2.9.3, pest_derive 2.9.3, pest_generator 2.9.3, pest_meta 2.9.3, petgraph 0.8.3, proteus 0.3.0 (path), psm 0.1.24, salsa20 0.10.2, sha256 1.6.0, stacker 0.1.25, supertunnel 0.1.0, thiserror 1.0.69 and thiserror-impl 1.0.69 (second versions), tokio-stream 0.1.19, ucd-trie 0.1.7, which 8.0.6 (build dependency of `proteus`) and xxhash-rust 0.8.19. `chacha20poly1305 0.10.1`, `x25519-dalek 2.0.1` and `rand 0.9.5` were already present. Of the Proteus dependencies only `chacha20poly1305`, `salsa20`, `sha256`, `pest`, `pest_derive`, `petgraph`, `bytes`, `thiserror`, `anyhow`, `hex`, `itertools`, `enum-from`, `tokio` and `tokio-util` lie on the code path the server reaches; `fast-socks5`, `supertunnel` ↪ , `asn1`, `pem`, `clap`, `env_logger`, `rand` and `x25519-dalek` serve the unreached `net`, `cli` and public-key PSF paths (`src/proteus_carrier.rs:15-21`, `src/bin/client.rs:11-14`). No dependency on the server path makes network connections of its own; the vendored crate's `build.rs` only probes the PATH for `python3`, `shadow`, `tgen` and `tor` to gate its system tests.

Crate	Version	Role	Advisory
tokio / tokio-util	1.53.1 / 0.7.19	runtime, codecs	none
snow	0.9.6	Noise NK	none
x25519-dalek	2.0.1	key derivation	none
yamux	0.13.10	MUX2	none
rustls	0.23.42	TLS client to Valkey	RUSTSEC-2026-0285 (5.3), -0346 (4.0), -0348 (3.7), -0349 (3.7); fixed in 0.23.46
tokio-rustls / rustls-pemfile	0.26.4 / 2.2.0	TLS adapter, PEM	rustls-pemfile: RUSTSEC-2025-0134 (unmaintained)
ring	0.17.14	Ed25519 verify, TLS provider	none
hickory-resolver	0.26.1	DNS	none
proteus	0.3.0 (path)	PSF compiler and interpreter	none in the database; RVPN-01-204, -206, -213, -214
chacha20poly1305 / salsa20 / sha256	0.10.1 / 0.10.2 / 1.6.0	PSF cipher, nonce stream, KDF	none
pest / petgraph	2.9.3 / 0.8.3	PSF grammar, task graph	none
serde, serde_json, toml, clap, log, env_logger, anyhow, chrono, mimalloc, socket2, libc, flate2, idna, futures, base64, hex, sha2, rand 0.8.7	see Cargo.lock	utilities	none

`cargo audit 0.22.2` against advisory database commit `edb58f5974b83eb8302332821cb6d389777aa351`: 4 vulnerabilities, 1 warning, identical to the WP1 result. No yanked crate was reported. The tree contains no SBOM document (CycloneDX 1.7 or SPDX 3.0.1) and no build provenance (SLSA v1.0); `Cargo.lock` and the vendored sources serve as the de facto inventory. The vendored Proteus crate's correspondence to upstream commit `75ba9f4` apart from the three documented edits could not be verified without network access. The crate is used under its own licence, reproduced at `PROTEUS-LICENSE` and `third_party/proteus/LICENSE` (identical files); its first paragraph reads, verbatim:

> To the extent that a federal employee is an author of a portion of this software or a derivative work thereof, no copyright is claimed by the United States Government, as represented by the Secretary of the Navy ("GOVERNMENT") under Title 17, U.S. Code. All Other Rights Reserved.

Red Team Ltd expresses no view on the licence's effect. For a product placed on the EU market, Regulation (EU) 2024/2847 Article 14 reporting duties apply from 11 September 2026; the vulnerability-handling process that the regulation expects, including an SBOM, is not evidenced in the tree.

Appendix C: Standards Conformance Checklist

OWASP ASVS 5.0.0, server-relevant chapters, cited at section level. Result values: Pass, Fail, Partial, Not assessed.

Control	Result	Note
V1.2 Injection Prevention	Pass	SOCKS, UDP, PROXY and RESP inputs are length-prefixed and bounded; statistics fields escaped for every line-break form (`src/stats.rs:121–129`)
V1.4 Memory, String and Unmanaged Code	Partial	No `unsafe` in the server or the vendored crate; unchecked arithmetic and `unwrap` in the interpreter (RVPN-01-204, -214)
V2.2 Input Validation	Partial	Prefix dispatch and PROXY parsing bounded; fixed fields not verified on receipt (RVPN-01-213); LOCAL/UNSPEC/UNKNOWN mishandled (RVPN-01-212)
V2.4 Anti-automation	Partial	5 s peek deadline, 600 s budget, 30 s grace and shedding exist; PROXY stage unbounded (RVPN-01-208); accept loop fragile (RVPN-01-210)
V6.3 General Authentication Security	Fail	No client authentication by design; cid asserted (RVPN-01-207)
V8.2 General Authorization Design	Fail	Mailbox, rules and usage keyed on the asserted cid (RVPN-01-207)
V11.3 Encryption Algorithms	Partial	Noise layer: pass. PSF layer: static key without salt, repeated nonces (RVPN-01-206)
V11.5 Random Values	Partial	Noise ephemeral keys from the CSPRNG; PSF randoms absent (RVPN-01-205)
V12.1 General TLS Security Guidance	Partial	Valkey connections CA-pinned with server-name verification; the imitated TLS of `tls_mimic.psf` is non-conformant (RVPN-01-205)

Control	Result	Note
V12.3 Service to Service Communication	Pass	Stats TLS; messaging TLS or mTLS; passwords sent after TLS
V13.1 Configuration Documentation	Pass	`proxy_protocol` default fixed and tested (RVPN-01-202); example configuration commented
V13.3 Secret Management	Partial	Password redacted from the banner (RVPN-01-203); placeholder password accepted (RVPN-01-215); example key and argv key (RVPN-01-219)
V13.4 Unintended Information Leakage	Partial	Nothing answered in clear (RVPN-01-201); `_rs.stat` and `_rs.config` (RVPN-01-209); behavioural tells (RVPN-01-211)
V14.2 General Data Protection	Partial	Statistics rows bind address, cid, SNI (RVPN-01-207); non-documentation addresses in fixtures (RVPN-01-218)
V15.2 Security Architecture and Dependencies	Partial	Vendored and locked; rustls advisories (RVPN-01-217); no SBOM
V15.3 Defensive Coding	Partial	Carrier and parsers bounded; interpreter unwraps (RVPN-01-204, -214)
V16.2 General Logging	Partial	Identities at `info` on the MUX2 path, PSF paths and per-connection lines (RVPN-01-216)
V16.5 Error Handling	Fail	Process abort on a failed tag (RVPN-01-204); accept-loop exit (RVPN-01-210)
V3, V4, V5, V7, V9, V10, V17	Not assessed	No web front end, file handling, sessions, tokens, OAuth or WebRTC in the component

Appendix D: Open Questions for the Maintainers

1. Is the production binary built with the `[profile.release]` of `Cargo.toml` (`panic = "abort"`)? Every test and script in the tree exercises the debug profile (RVPN-01-204).
2. Is the pairing of `tls_mimic.psf` and `shadowsocks.psf` in `config.example.toml` intended for public nodes? With it, ordinary TLS clients reach the shadowsocks interpreter.
3. Are the two shipped PSFs meant as fixtures for the end-to-end tests or as production carriers (RVPN-01-205, -206)?
4. Will the upstream Proteus project add per-connection salts and counter nonces, or will the vendored fork be maintained with the behavioural fixes of RVPN-01-204, -206, -213 and -214?
5. How will the shadowsocks password be distributed to and rotated in the mobile clients once they speak a PSF, and will it be shared across nodes, which makes the opener of RVPN-01-206 fleet-wide?
6. Which addresses and ports will the carrier listen on in production, and will a real HTTPS service answer on them, which would make the negative path of RVPN-01-211 realistic?
7. Should an `extra_binds` failure be fatal, as the primary listener's accept error is (RVPN-01-210)?
8. Is `[messaging]` planned for public exits running this server, and has the client-identity decision of RVPN-01-207 been taken?
9. Does the vendored tree match upstream commit `75ba9f4` apart from the three documented edits?

10. Are the publicly routable literals listed in RVPN-01-218 arbitrary fixtures or real addresses?
11. Is the 5 s silent-peer close a deliberate trade-off against slow-loris, or can it be aligned with a real server's idle timeout (RVPN-01-211)?